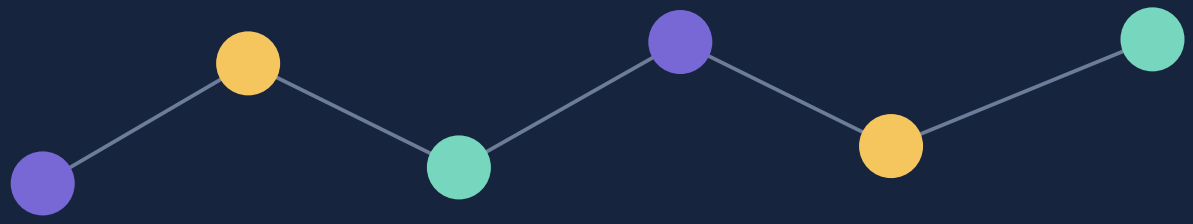
GATE 2027 / COMPLETE SYLLABUS EDITION

Algorithms

Start from the basics. Build every syllabus idea. Then practise.

Section 5: Algorithms



LEARN → SEE → WORK THROUGH → RETRIEVE → PRACTISE

1 Understand

Each chapter starts in plain language.

2 Visualize

Follow the flow and redraw it.

3 Work

Trace the worked example with pencil.

4 Recall

Answer before checking the solution.

A beginner learning guide + illustrated recall/practice + your supplied reference material

2027 syllabus coverage map

Use this as a route map: every item below has a beginner lesson in this PDF. The supplied handbook/chapter PDFs are gathered in the reference section after the learning and practice pages.

- [ ] **Worst-case time, space and asymptotic growth**

  Big-O, Omega, Theta • Worst case • Space • Recurrences

- [ ] **Searching and sorting**

  Linear and binary search • Sort families • Properties • Selection and bounds

- [ ] **Hashing**

  Hash function • Chaining • Open addressing • Load factor

- [ ] **Greedy algorithms**

  Greedy-choice property • Optimal substructure • Proof tools • When it fails

- [ ] **Dynamic programming**

  State • Recurrence and base cases • Memoization and tabulation • Reconstruction

- [ ] **Divide-and-conquer**

  Divide • Conquer • Combine • Analysis

- [ ] **Graph traversals, MSTs and shortest paths**

  BFS/DFS • Minimum spanning tree • Shortest paths • DAG paths

- [ ] **Solving recurrences and proving a greedy choice**

  Master theorem check • Uneven subproblems • Greedy exchange • Worst-case bound

- [ ] **Graph search, shortest paths and minimum spanning trees**

  BFS and distances • DFS and components • MST cut property • Shortest paths

Your route through this subject

Follow the lessons in order. Tick a topic only after you can explain its picture without looking; a tick means “I can teach it,” not “I have seen it.”

1. **Worst-case time, space and asymptotic growth**
2. **Searching and sorting**
3. **Hashing**
4. **Greedy algorithms**
5. **Dynamic programming**
6. **Divide-and-conquer**
7. **Graph traversals, MSTs and shortest paths**
8. **Solving recurrences and proving a greedy choice**
9. **Graph search, shortest paths and minimum spanning trees**

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	**Redraw**	**Solve**	**Recall**

IF THIS SUBJECT FEELS COMPLETELY NEW

That is okay. Start with the first lesson, keep a pencil beside you, and redraw its picture once. Attempt the recall prompt before opening the answer guide.

Worst-case time, space and asymptotic growth

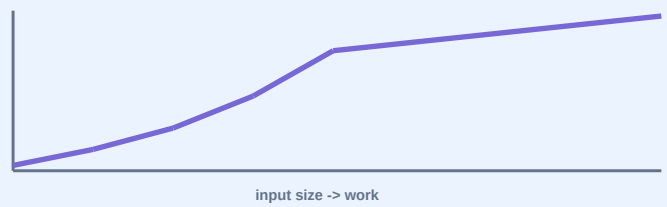
START WITH THE IDEA

Algorithm analysis predicts how resource use grows with input size, without depending on one computer's clock speed.

A CONCRETE FIRST EXAMPLE

3n^2+5n+2 is Theta(n^2). For T(n)=2T(n/2)+n, the two recursive halves plus linear merge give Theta(n log n).

PICTURE IT



IMAGINE IT THIS WAY

Complexity is the growth pattern of a recipe as the guest count grows.

THE IDEAS, ONE BY ONE

Big-O, Omega, Theta

O is an asymptotic upper bound; Omega a lower bound; Theta a tight bound when both apply.

Worst case

Ask for the maximum work over all inputs of size n; average case requires an input distribution.

Space

Count extra storage and recursive call stack, not just the input, using the question's convention.

Recurrences

Recursive algorithms produce equations such as T(n)=2T(n/2)+n; solve with recursion trees or Master theorem when assumptions fit.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Do not apply Master theorem outside its permitted recurrence form.

TRY FROM MEMORY

Cover this page. Explain Worst-case time, space and asymptotic growth using the picture, then give one example before looking again.

Searching and sorting

START WITH THE IDEA

Searching locates a key; sorting arranges keys. Correctness and worst-case guarantees matter alongside speed and memory.

A CONCRETE FIRST EXAMPLE

Binary search on 64 sorted values takes at most 7 comparisons. Merge sort combines two sorted halves in linear time at each level.

PICTURE IT



IMAGINE IT THIS WAY

Binary search is an index that discards half the candidates after each clue.

THE IDEAS, ONE BY ONE

Linear and binary search

Linear scan works on any sequence in O(n); binary search halves a sorted search interval in O(log n).

Sort families

Insertion/selection/bubble are simple O(n^2) worst case; merge sort is Theta(n log n); quicksort is expected Theta(n log n), worst-case Theta(n^2).

Properties

Stable sort preserves equal-key order. In-place sort uses small extra memory, subject to implementation details.

Selection and bounds

Comparison sorting requires Omega(n log n) comparisons in the general case; counting/radix use assumptions about key ranges/digits.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Always include sorted-input assumptions when choosing binary search.

TRY FROM MEMORY

Cover this page. Explain Searching and sorting using the picture, then give one example before looking again.

Hashing

START WITH THE IDEA

Hashing maps a key to a table position for fast expected lookup. Collisions are unavoidable when many possible keys share fewer slots.

A CONCRETE FIRST EXAMPLE

If h(k)=k mod 7, keys 10 and 17 both reach bucket 3. Chaining stores both there; linear probing searches later slots.

PICTURE IT



IMAGINE IT THIS WAY

A hash table is a row of mailboxes; the function chooses a box but does not promise uniqueness.

THE IDEAS, ONE BY ONE

Hash function

h(key) selects a bucket or initial probe position; distribution quality matters.

Chaining

Store a list or collection at each bucket. Lookup cost depends on load and hash quality.

Open addressing

Keep all keys inside the table and probe on collision: linear, quadratic or double hashing.

Load factor

alpha=n/m is keys per slot on average. Higher load usually means more collisions and longer probes.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Simulate the exact probe rule and wrap modulo the table size.

TRY FROM MEMORY

Cover this page. Explain Hashing using the picture, then give one example before looking again.

Greedy algorithms

START WITH THE IDEA

Greedy algorithms make a locally attractive choice and keep it. This is safe only when a proof shows that some optimal solution can include that choice.

A CONCRETE FIRST EXAMPLE

For activity selection, choose the compatible activity that finishes earliest; it leaves maximum time for remaining activities.

PICTURE IT



IMAGINE IT THIS WAY

Greedy picks one ticket now; prove it cannot close off the best future plan.

THE IDEAS, ONE BY ONE

Greedy-choice property

A locally best first choice can be extended to an optimal complete answer.

Optimal substructure

After making the safe choice, the remainder is an instance of the same problem.

Proof tools

Exchange arguments replace part of an optimal solution; cut properties justify choices in MSTs.

When it fails

For arbitrary coin values or 0/1 knapsack, the locally largest/immediate reward choice may block a better combination.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT A few successful examples are not a correctness proof.

TRY FROM MEMORY

Cover this page. Explain Greedy algorithms using the picture, then give one example before looking again.

Dynamic programming

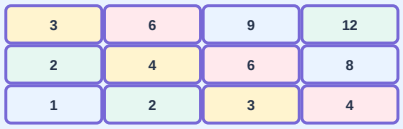
START WITH THE IDEA

DP solves a problem by defining smaller states, reusing overlapping subproblem answers and building an optimal answer from them.

A CONCRETE FIRST EXAMPLE

LCS state dp[i][j] for prefixes: match ends -> 1+dp[i-1][j-1]; otherwise max(dp[i-1][j],dp[i][j-1]).

PICTURE IT



small answers build larger answers

IMAGINE IT THIS WAY

DP is a notebook: solve each distinct subproblem once and reuse the answer.

THE IDEAS, ONE BY ONE

State

Write exactly what dp[i] or dp[i][j] means in words.

Recurrence and base cases

Express each state using smaller states; specify initial states and unreachable cases.

Memoization and tabulation

Memoization caches recursive results; tabulation computes in dependency order.

Reconstruction

Store decisions or follow equalities backward when the actual solution—not only its value—is requested.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT A recurrence without a state meaning, base cases and evaluation order is incomplete.

TRY FROM MEMORY

Cover this page. Explain Dynamic programming using the picture, then give one example before looking again.

Divide-and-conquer

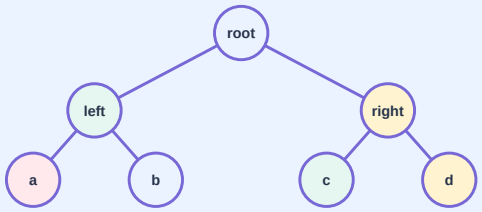
START WITH THE IDEA

Divide-and-conquer splits a problem into smaller mostly independent instances, solves them recursively, and combines their results.

A CONCRETE FIRST EXAMPLE

Merge sort has 2 subproblems of size n/2 and Theta(n) merge work: T(n)=2T(n/2)+Theta(n)=Theta(n log n).

PICTURE IT



IMAGINE IT THIS WAY

Split a pile, solve each smaller pile, and pay for putting the results back together.

THE IDEAS, ONE BY ONE

Divide
Identify number and size of subproblems.

Conquer
Solve recursively, stopping at a simple base case.

Combine
Account for merge or aggregation cost; this work may dominate.

Analysis
T(n)=aT(n/b)+f(n) models a subproblem count a, shrink factor b and combine work f(n).

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Include recursion-stack space when asked for total space.

TRY FROM MEMORY

Cover this page. Explain Divide-and-conquer using the picture, then give one example before looking again.

Graph traversals, MSTs and shortest paths

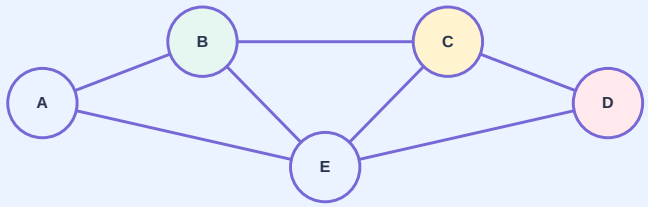
START WITH THE IDEA

Graph problems ask different questions: visit vertices, connect all vertices cheaply, or find low-cost routes. Matching the algorithm to the objective is essential.

A CONCRETE FIRST EXAMPLE

An MST minimizes total network cable length; it need not give the shortest path from a chosen root to every vertex.

PICTURE IT



IMAGINE IT THIS WAY

First ask: reachability, cheapest all-vertex connection, or source-to-destination route?

THE IDEAS, ONE BY ONE

BFS/DFS

BFS explores by unweighted distance; DFS explores deeply and backtracks.

Minimum spanning tree

Kruskal and Prim connect all vertices with minimum total edge weight, assuming a connected undirected graph.

Shortest paths

BFS for unweighted edges; Dijkstra for nonnegative weights; Bellman–Ford allows negative edges and detects reachable negative cycles.

DAG paths

Topological order supports efficient shortest/longest path dynamic programming in a directed acyclic graph.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Dijkstra's usual correctness guarantee requires nonnegative edge weights.

TRY FROM MEMORY

Cover this page. Explain Graph traversals, MSTs and shortest paths using the picture, then give one example before looking again.

Solving recurrences and proving a greedy choice

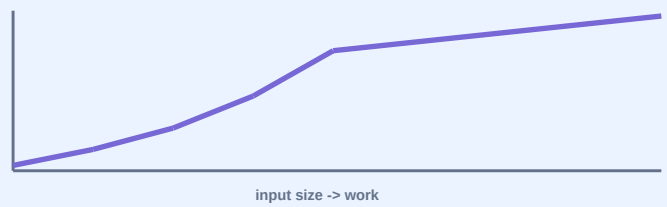
START WITH THE IDEA

An algorithm question asks for a bound and a reason it is correct. Recurrences describe recursive work; greedy proofs explain why a local choice can be kept in an optimum.

A CONCRETE FIRST EXAMPLE

T(n)=2T(n/2)+n has level cost n and log2(n) levels, so T(n)=Theta(n log n).

PICTURE IT



IMAGINE IT THIS WAY

A recurrence tree is a bill for each level of recursive work; a greedy proof swaps one choice at a time.

THE IDEAS, ONE BY ONE

Master theorem check

For T(n)=aT(n/b)+f(n), compare f(n) with n^(log_b a). Check regularity before selecting a case.

Uneven subproblems

Expand one or two levels or use a recursion tree when subproblem sizes differ or the recurrence has nested logarithms.

Greedy exchange

Start with an optimal solution, replace its first choice by the greedy choice, and show the replacement does not worsen the solution.

Worst-case bound

Count the operation that can dominate on the hardest valid input; state time and extra-space assumptions.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT Do not apply the Master theorem when the recurrence does not match its required form.

TRY FROM MEMORY

Cover this page. Explain Solving recurrences and proving a greedy choice using the picture, then give one example before looking again.

Graph search, shortest paths and minimum spanning trees

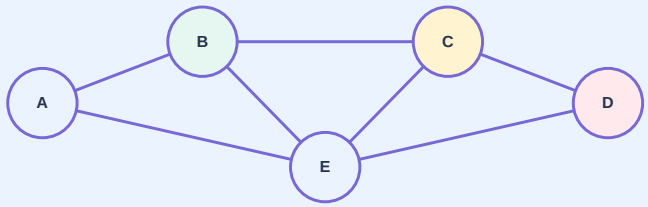
START WITH THE IDEA

BFS explores in layers, DFS follows a path deeply, and an MST connects all vertices with minimum total edge weight. These solve different graph problems.

A CONCRETE FIRST EXAMPLE

If every edge weight is distinct, the MST is unique. If weights tie, several spanning trees can have the same minimum total.

PICTURE IT



IMAGINE IT THIS WAY

BFS is a ripple of water; DFS is exploring a maze corridor. An MST is the cheapest set of pipes joining all towns.

THE IDEAS, ONE BY ONE

BFS and distances

In an unweighted graph, BFS discovers vertices in nondecreasing number of edges from the source.

DFS and components

DFS marks a vertex when first reached. Restart from each unvisited vertex to find connected components.

MST cut property

For any cut, a lightest crossing edge is safe for some MST. Distinct weights give a unique MST.

Shortest paths

Dijkstra requires nonnegative edge weights. Bellman-Ford can handle negative edges if there is no reachable negative cycle.

HOW TO USE IT ON A PROBLEM

State input, output and worst-case assumptions. Choose a strategy, trace a small instance, count time/space, then give the correctness reason and edge case.

WATCH OUT An MST minimizes the total tree weight; it does not necessarily contain each pair's shortest path.

TRY FROM MEMORY

Cover this page. Explain Graph search, shortest paths and minimum spanning trees using the picture, then give one example before looking again.

Past-paper trail and coverage check

WHAT THE PAPERS MATCH

Verified anchors: 2018 Q31/Q45/Q47-Q48 (dynamic programming, recursion, MST and knapsack); 2022 Q11/Q16/Q49/Q51 (complexity, hashing, MST and recurrence); 2023 Q14/Q19/Q28/Q45/Q53 (recurrence, hashing, asymptotic growth and heaps); 2024 CS1 Q17/Q34/Q42/Q45; 2025 CS1 Q18/Q20/Q33/Q43/Q64-Q65 and CS2 Q20/Q29/Q37/Q41/Q59; 2026 CS1 Q17/Q24/Q41/Q49-Q50 and CS2 Q24-Q25/Q32/Q36-Q39.

WHAT THIS EDITION ADDS

Added recurrence solving, greedy exchange intuition, BFS layers, DFS components, MST cut property, shortest-path assumptions and hashing traces.

SYLLABUS BOUNDARY

Amortized analysis, SCC, articulation/bridges and max-flow are not explicitly listed in the supplied 2027 Algorithms syllabus; treat as optional extensions unless an official syllabus update adds them.

HOW TO USE THIS CHECK

A question is a match when its tested skill is taught in the lesson pages, even if the story or numbers are new. Anchors are selected verified examples, not an exhaustive list; the separate Coverage and Gap Audit PDF identifies years whose scans could not be checked question by question.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

1. **Worst-case time, space and asymptotic growth**

   Without notes: define Recurrences and explain why this example works. Then recreate the steps on the lesson page.

2. **Searching and sorting**

   Without notes: define Selection and bounds and explain why this example works. Then recreate the steps on the lesson page.

3. **Hashing**

   Without notes: define Load factor and explain why this example works. Then recreate the steps on the lesson page.

4. **Greedy algorithms**

   Without notes: define When it fails and explain why this example works. Then recreate the steps on the lesson page.

5. **Dynamic programming**

   Without notes: define Reconstruction and explain why this example works. Then recreate the steps on the lesson page.

6. **Divide-and-conquer**

   Without notes: define Analysis and explain why this example works. Then recreate the steps on the lesson page.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

7 **Graph traversals, MSTs and shortest paths**

Without notes: define DAG paths and explain why this example works. Then recreate the steps on the lesson page.

8 **Solving recurrences and proving a greedy choice**

Without notes: define Worst-case bound and explain why this example works. Then recreate the steps on the lesson page.

9 **Graph search, shortest paths and minimum spanning trees**

Without notes: define Shortest paths and explain why this example works. Then recreate the steps on the lesson page.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

1. **LESSON 1: Worst-case time, space and asymptotic growth**

   3n^2+5n+2 is Theta(n^2). For T(n)=2T(n/2)+n, the two recursive halves plus linear merge give Theta(n log n).

2. **LESSON 2: Searching and sorting**

   Binary search on 64 sorted values takes at most 7 comparisons. Merge sort combines two sorted halves in linear time at each level.

3. **LESSON 3: Hashing**

   If h(k)=k mod 7, keys 10 and 17 both reach bucket 3. Chaining stores both there; linear probing searches later slots.

4. **LESSON 4: Greedy algorithms**

   For activity selection, choose the compatible activity that finishes earliest; it leaves maximum time for remaining activities.

5. **LESSON 5: Dynamic programming**

   LCS state dp[i][j] for prefixes: match ends -> 1+dp[i-1][j-1]; otherwise max(dp[i-1][j],dp[i][j-1]).

6. **LESSON 6: Divide-and-conquer**

   Merge sort has 2 subproblems of size n/2 and Theta(n) merge work: T(n)=2T(n/2)+Theta(n)=Theta(n log n).

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

7. LESSON 7: Graph traversals, MSTs and shortest paths

An MST minimizes total network cable length; it need not give the shortest path from a chosen root to every vertex.

8. LESSON 8: Solving recurrences and proving a greedy choice

T(n)=2T(n/2)+n has level cost n and log2(n) levels, so T(n)=Theta(n log n).

9. LESSON 9: Graph search, shortest paths and minimum spanning trees

If every edge weight is distinct, the MST is unique. If weights tie, several spanning trees can have the same minimum total.

Your practice and reference library

01 **Beginner lessons**

Plain-language explanation, visual sequence, example and watch-outs for every listed 2027 topic cluster.

02 **Recall and GATE-style practice**

The illustrated recap section includes questions and a separate answer explanation.

03 **Detailed reference**

Your supplied subject handbook and chapter PDFs follow for extra depth, reading and worked material.

Included source documents

Illustrated recall and GATE-style practice AL - Algorithms - GATE 2027 Illustrated Notes.pdf	12 pages
Detailed supplied reference AL.pdf	37 pages
Detailed supplied reference GATE2027_Chapter5_DSA4_Graphs.pdf	5 pages
Detailed supplied reference GATE2027_Chapter6_Algorithms_Sorting_Hashing_Greedy.pdf	4 pages
Detailed supplied reference GATE2027_Chapter7_Dynamic_Programming.pdf	3 pages

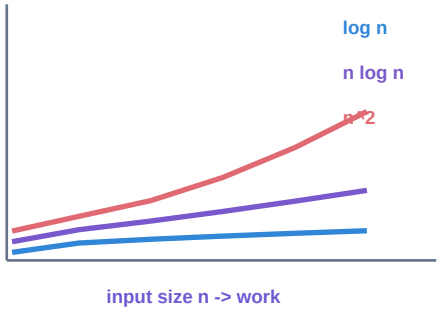
The official 2027 syllabus map precedes the lessons. Original reference pages are retained so you can use them for slower reading and extra practice.

Algorithms

GATE 2027 | VISUAL LEARNING NOTES | STARTING FROM THE BASICS

Choose a strategy. Prove its bound. Test the edge cases.

PICTURE IT



HOW TO USE THIS NOTEBOOK

1. **Learn the idea**
   Read the plain-language explanation before memorizing a formula.
2. **See it**
   Study the labeled picture and redraw it from memory.
3. **Follow a solve**
   Work through the examples step by step.
4. **Test yourself**
   Attempt the GATE-style questions before opening the key.

THE PATH YOU WILL FOLLOW

- Core methods
- Design techniques
- Graph algorithms

IF YOU FEEL COMPLETELY NEW

That is okay. Start with the first topic page, keep a pencil beside you, and copy each diagram once. Do not jump straight to the answer key. The questions are meant to come after the concepts.

Your route through this subject

Follow the topics in order. Each page teaches one idea, shows a picture, gives a recall cue and points out a common trap.

CORE METHODS

- [ ] Searching, sorting and hashing.
- [ ] Asymptotic worst-case time and space complexity.

DESIGN TECHNIQUES

- [ ] Greedy algorithms.
- [ ] Dynamic programming.
- [ ] Divide-and-conquer.

GRAPH ALGORITHMS

- [ ] Graph traversals.
- [ ] Minimum spanning trees.
- [ ] Shortest paths.

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

Tick a topic only after you can explain its picture without looking. A tick means "I can teach it," not "I have seen it."

BEFORE EACH STUDY SESSION

Pick one topic. Read its idea, redraw its diagram, follow the worked examples, then answer questions with the notes closed. Circle only the step where you got stuck - that is your next revision target.

CONCEPT 01 / AL

Asymptotic bounds and recurrences

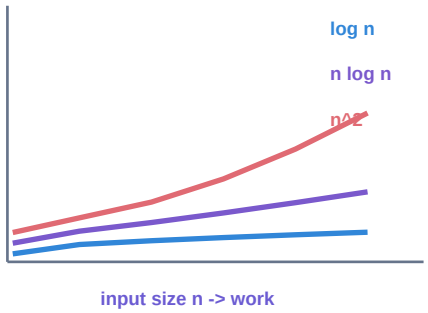
START WITH THE IDEA

O is an upper bound, Omega a lower bound, Theta a tight bound. Ignore constant factors for growth. Recurrences describe recursive algorithms; use a recursion tree or Master theorem when its conditions fit.

A CONCRETE FIRST EXAMPLE

Asymptotic notation describes how work grows with input size, ignoring constant factors and lower-order terms. A recurrence expresses an algorithm's time using smaller inputs. 3n^2+5n+2 is Theta(n^2). Doubling n eventually makes the quadratic term about four times as large.

PICTURE IT



IMAGINE IT THIS WAY

Count how a job grows when the input doubles; that trend matters more than seconds on one laptop.

HOW TO USE IT ON A PROBLEM

Name the input size and worst case before simplifying.

WATCH OUT

Do not apply the Master theorem to a recurrence that does not match its form.

TRY FROM MEMORY

Cover the page and explain Asymptotic bounds and recurrences using the diagram. Give one example before looking again.

CONCEPT 02 / AL

Searching and sorting

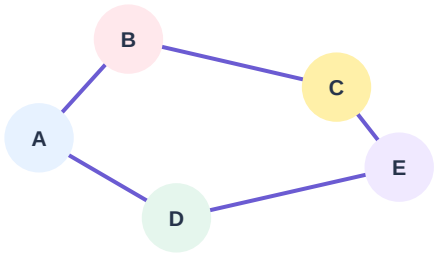
START WITH THE IDEA

Binary search needs sorted data and halves the search interval. Merge sort is O(n log n) worst-case; insertion sort can be O(n^2) but adaptive on nearly sorted inputs. Partition-based sorts have expected O(n log n) but a quadratic worst case in poor pivots.

A CONCRETE FIRST EXAMPLE

Searching locates a key; sorting orders records. Binary search repeatedly halves a sorted range, so it needs only logarithmically many comparisons. Sorting algorithms differ in stability, memory and worst-case time. In 64 sorted items, binary search needs at most 7 comparisons because 2■<64<=2■.

PICTURE IT



compare, then shrink the search space

IMAGINE IT THIS WAY

Binary search is a book index: discard half the pages at each comparison.

HOW TO USE IT ON A PROBLEM

Check stability, in-place space and worst-case assumptions separately.

WATCH OUT

A sorted output does not reveal whether the algorithm is stable.

TRY FROM MEMORY

Cover the page and explain Searching and sorting using the diagram. Give one example before looking again.

CONCEPT 03 / AL

Hashing

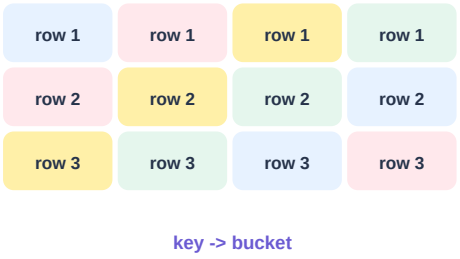
START WITH THE IDEA

A hash function maps keys to table positions; collisions need chaining or probing. Load factor alpha=n/m influences expected probe work. Linear probing can cluster; double hashing uses a second step size.

A CONCRETE FIRST EXAMPLE

Hashing turns a key into a table index. Different keys can collide; chaining stores a list per bucket, while open addressing searches for another slot. Load factor affects expected performance. If h(k)=k mod 7, keys 10 and 17 both map to bucket 3; collision handling must keep both findable.

PICTURE IT



IMAGINE IT THIS WAY

A hash table is a row of mailboxes; two names can still land at one box.

HOW TO USE IT ON A PROBLEM

Simulate the exact probe sequence and wrap with modulo table size.

WATCH OUT

A good average-case claim is not a worst-case guarantee.

TRY FROM MEMORY

Cover the page and explain Hashing using the diagram. Give one example before looking again.

CONCEPT 04 / AL

Greedy choice

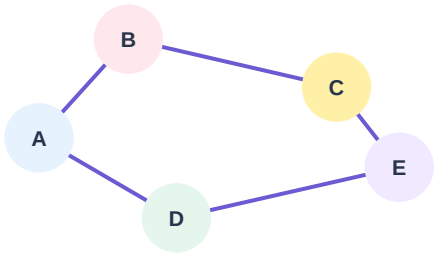
START WITH THE IDEA

A greedy algorithm commits to a locally best choice. It is correct only when an exchange argument or cut/property proof supports that choice. Examples include interval scheduling and Kruskal/Prim for MST.

A CONCRETE FIRST EXAMPLE

A greedy algorithm commits to a locally attractive choice. It is correct only when the problem has a proof structure, often an exchange argument showing an optimal solution can start with that choice. For activity selection, choosing the activity that finishes earliest leaves the largest remaining time window.

PICTURE IT



choose a safe local move

IMAGINE IT THIS WAY

Pick the next best ticket only when you can prove no optimal plan loses by it.

HOW TO USE IT ON A PROBLEM

State the greedy rule and prove why a safe exchange exists.

WATCH OUT

A plausible greedy rule can fail for 0/1 knapsack.

TRY FROM MEMORY

Cover the page and explain Greedy choice using the diagram. Give one example before looking again.

CONCEPT 05 / AL

Dynamic programming

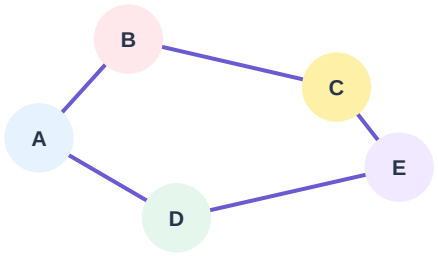
START WITH THE IDEA

DP applies overlapping subproblems and optimal substructure. Define state, recurrence, base cases, evaluation order and answer location. Memoization is top-down; tabulation is bottom-up.

A CONCRETE FIRST EXAMPLE

Dynamic programming saves answers to overlapping subproblems. Define a state, recurrence, base cases and evaluation order; then reconstruct a solution if the question asks for the choices too. Fibonacci: F(0)=0,F(1)=1,F(n)=F(n-1)+F(n-2). A table computes each value once instead of repeating calls.

PICTURE IT



reuse solved subproblems

IMAGINE IT THIS WAY

DP is a notebook: solve each distinct subproblem once and reuse the result.

HOW TO USE IT ON A PROBLEM

Write the state meaning in words before writing the recurrence.

WATCH OUT

A recurrence without base cases or a dependency order is incomplete.

TRY FROM MEMORY

Cover the page and explain Dynamic programming using the diagram. Give one example before looking again.

CONCEPT 06 / AL

Divide-and-conquer

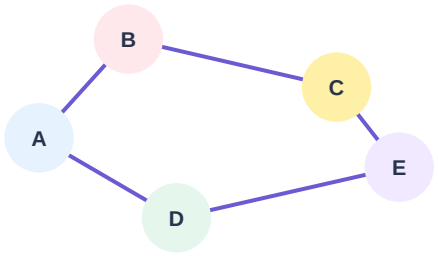
START WITH THE IDEA

Divide into smaller independent pieces, solve recursively, combine. Analyze the recurrence, including merge/combine cost and recursion stack.

A CONCRETE FIRST EXAMPLE

Divide and conquer splits a problem into smaller independent pieces, solves them recursively, then combines results. The recurrence captures both the subproblem work and the combine step. Merge sort uses T(n)=2T(n/2)+Theta(n), giving Theta(n log n) work.

PICTURE IT



split -> solve -> combine

IMAGINE IT THIS WAY

Split a large pile into smaller piles, then combine sorted results.

HOW TO USE IT ON A PROBLEM

For T(n)=aT(n/b)+f(n), compare f(n) with n^(log_b a).

WATCH OUT

Do not omit the combine step or stack space.

TRY FROM MEMORY

Cover the page and explain Divide-and-conquer using the diagram. Give one example before looking again.

CONCEPT 07 / AL

Graph algorithms

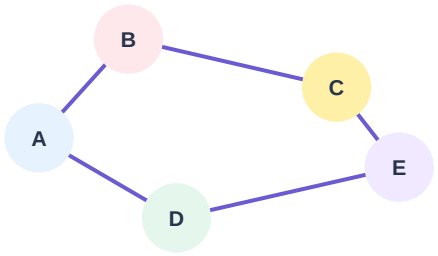
START WITH THE IDEA

BFS/DFS traverse; Kruskal/Prim build an MST; Dijkstra requires nonnegative edge weights; Bellman-Ford handles negative edges and can detect a reachable negative cycle.

A CONCRETE FIRST EXAMPLE

Graph algorithms depend on the question: BFS handles unweighted shortest paths, Dijkstra handles nonnegative weights, and Kruskal/Prim build a minimum spanning tree. A path and a spanning tree solve different tasks. If an edge has negative weight, Dijkstra's assumptions fail; do not apply it blindly.

PICTURE IT



choose the cheapest valid path

IMAGINE IT THIS WAY

Routing is a map problem: choose whether you need reachability, a cheap spanning network or one-source distances.

HOW TO USE IT ON A PROBLEM

Match algorithm assumptions to edge signs, direction and requested output.

WATCH OUT

An MST is not a shortest-path tree.

TRY FROM MEMORY

Cover the page and explain Graph algorithms using the diagram. Give one example before looking again.

Let's solve it together

Cover the answer with your hand. Try each step first, then compare your reasoning.

EXAMPLE 1: Master theorem

QUESTION

Solve T(n)=2T(n/2)+n with T(1)=1.

WORK THROUGH THE STEPS

a=2,b=2, so n^(log_b a)=n. f(n)=n matches the

ANSWER Theta(n log n).

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

EXAMPLE 2: Greedy vs DP

QUESTION

For coin values {1,3,4}, make 6 with fewest coins.

WORK THROUGH THE STEPS

Greedy takes 4+1+1 (3 coins). DP checks

ANSWER Greedy is not generally optimal for arbitrary coin systems; DP returns 2 coins here.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

Now you try

Attempt these without the notes. The question types are marked MCQ, MSQ or NAT. Answers and explanations are on the next page.

MCQ Binary search on 64 sorted items needs at most how many comparisons in the standard worst case?

(A) 6 (B) 7 (C) 32 (D) 64

NAT For T(n)=2T(n/2)+n, what is the asymptotic time?

Use a tight Theta bound.

MCQ Which shortest-path algorithm is valid with negative edges, assuming no reachable negative cycle?

(A) Dijkstra (B) Bellman-Ford (C) BFS (D) Prim

MSQ Which statements are true?

(A) MST minimizes total tree-edge weight (B) MST necessarily gives shortest paths from a root (C) Merge sort worst case is O(n log n) (D) DP requires overlapping subproblems and optimal substructure

NAT For strings ABC and AC, what is the LCS length?

Longest common subsequence; characters need not be adjacent.

Exam habit: decide what is being asked before doing arithmetic. Show units, assumptions and one line of reasoning.

Answers with the why

Read the explanation even when your answer is correct. The goal is to learn the reasoning pattern.

1. B

WHY

ceil(log2(64+1))=7 comparisons in the worst case.

2. Theta(n log n)

WHY

The merge cost matches n^(log_2 2)=n, giving the critical Master-theorem case.

3. B

WHY

Bellman-Ford supports negative edge weights and detects reachable negative cycles.

4. A, C, D

WHY

An MST is not necessarily a shortest-path tree.

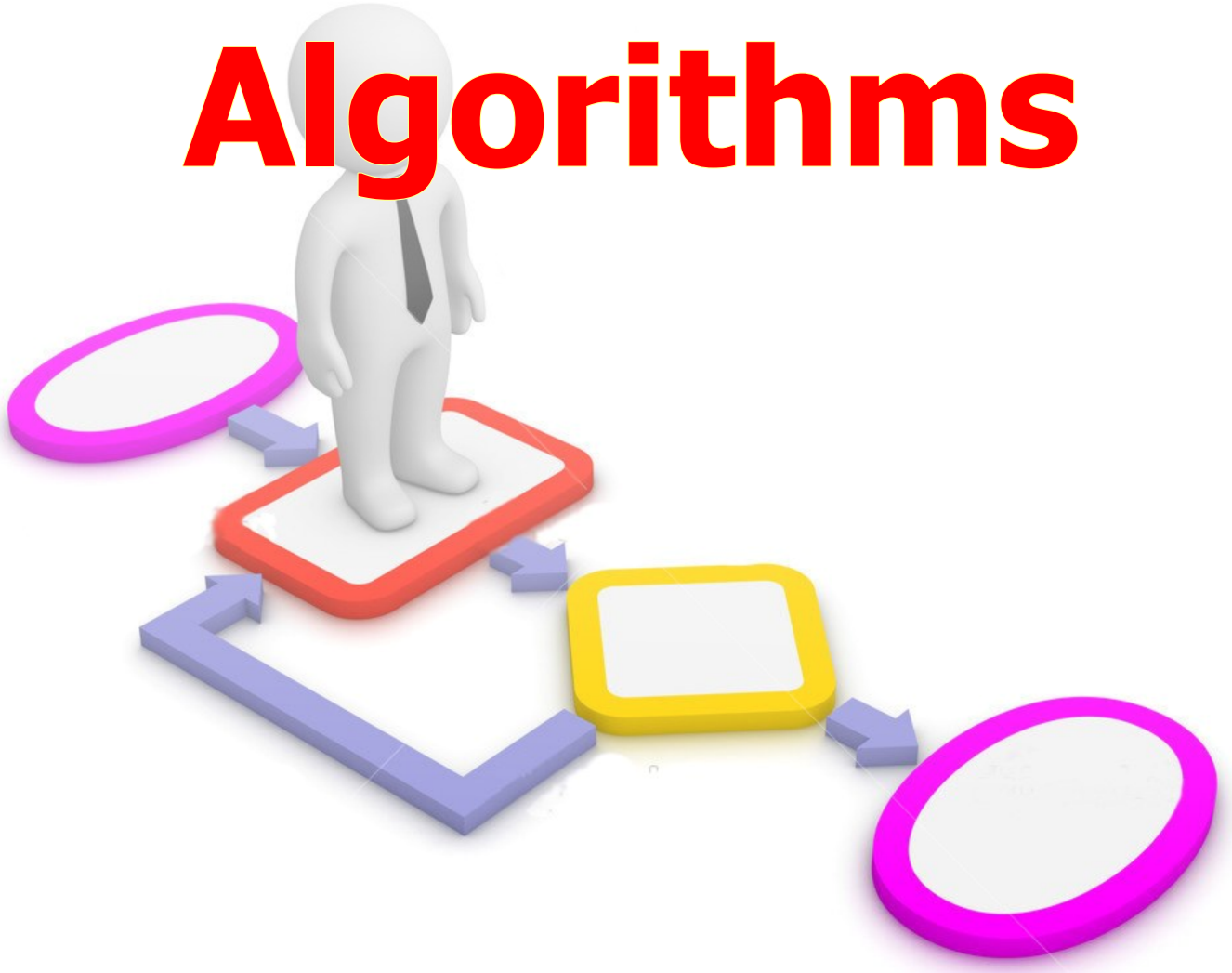
5. 2

WHY

AC is a common subsequence of length 2; the shorter string has length 2.

Review loop: mark each missed concept page, redraw its picture, and retry a similar problem tomorrow.

Algorithms



Published By:

Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)

Website: www.pw.live

Email: support@pw.live

Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

This book and the individual contribution contained in it are protected under copyright by the publisher.

(This Module shall only be Used for Educational Purpose.)

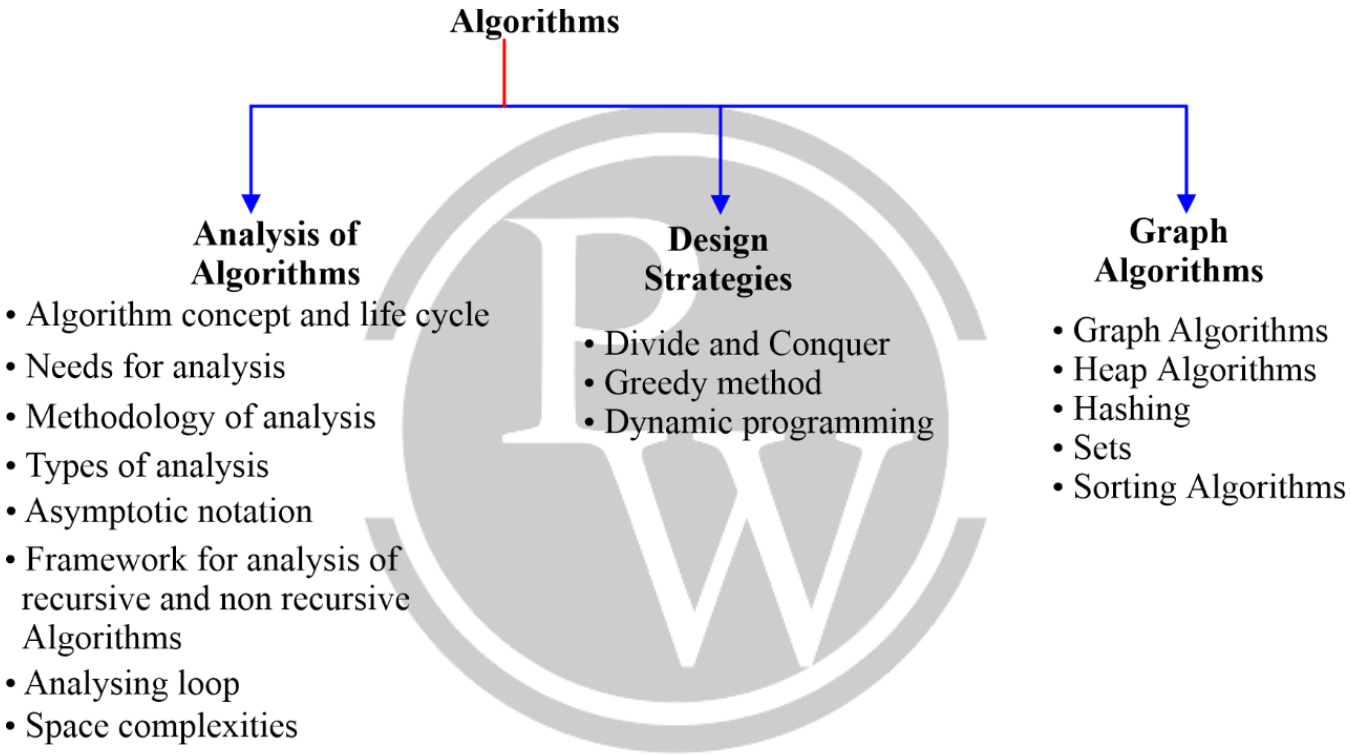
Algorithm

INDEX

1. Asymptotic Notation ..... 6.1 – 6.18
2. Divide and Conquer ..... 6.19 – 6.25
3. Greedy Technique ..... 6.26 – 6.29
4. Dynamic Programming ..... 6.30 – 6.34

1 ASYMPTOTIC NOTATION

1.1 Introduction of Course

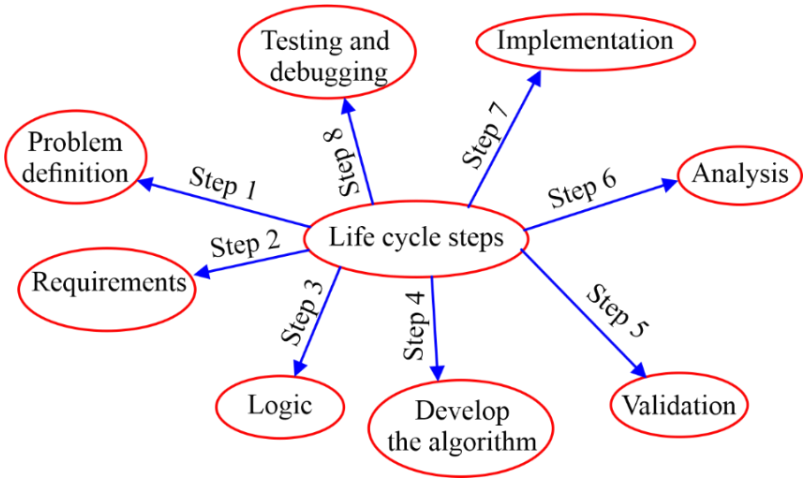


1.2 Algorithm Concept and Life Cycle Steps

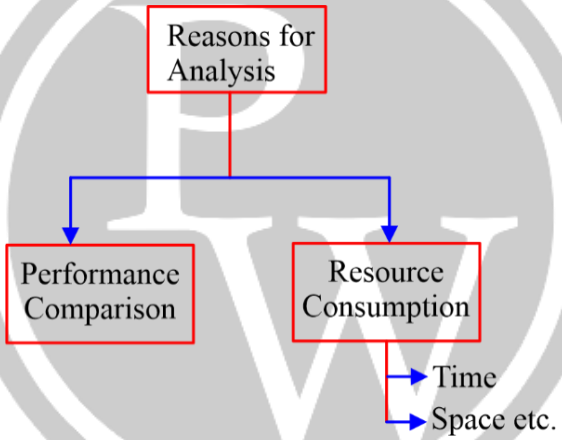
1.2.1 Algorithm

- An Algorithm consists finite number of steps to solve any problem.
- Every step involves some operations and each operation must be definite and effective.

1.2.2 Life Cycle Steps



1.3 Needs of Analysis



In performance comparison comparing different algorithms for optimal solution.

1.3.1 Time Complexity

Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the input size.

1.3.2 Space Complexity

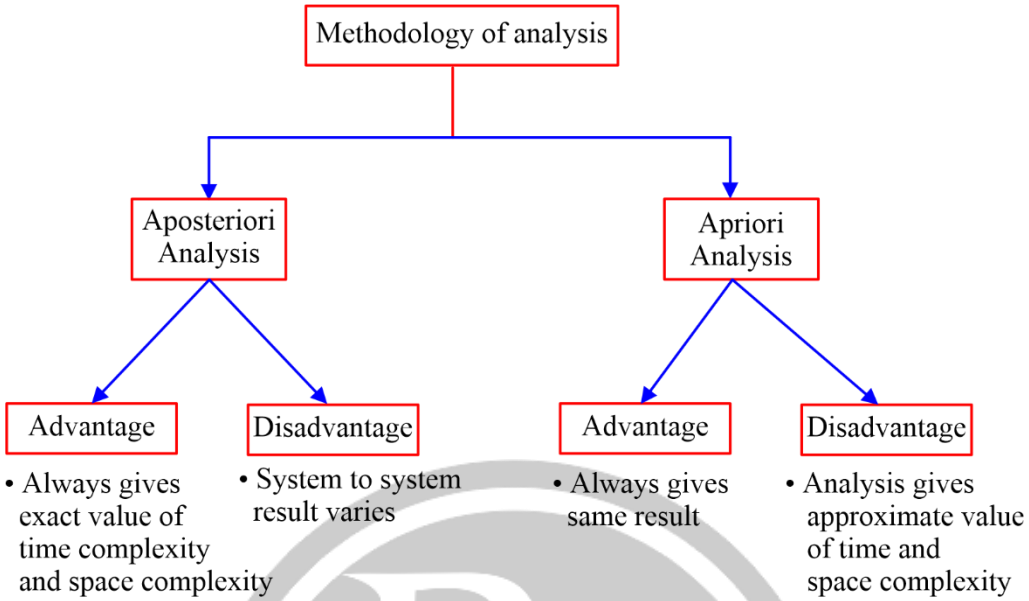
Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of input size.

Note:

To find the time complexity of an algorithm, find the loops and also consider larger loops.

Space complexity is dependent on two things input size and some extra space (stack space link, space list etc).

1.4 Methodology of Analysis



1.5 Types of Analysis

Worst Case

The input class for which the algorithm does maximum work and hence, take maximum time.

Best Case

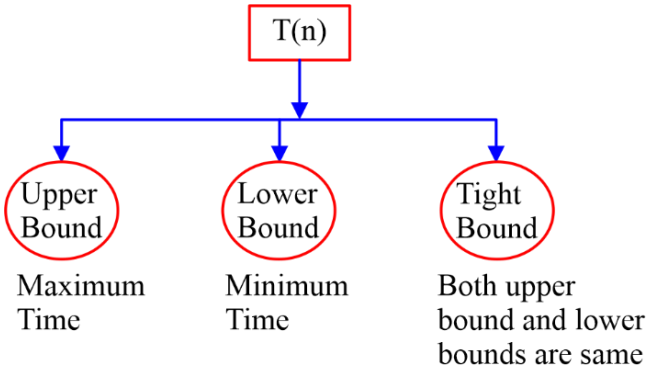
The input class for which the algorithm does minimum work hence, take minimum time.

Average Case

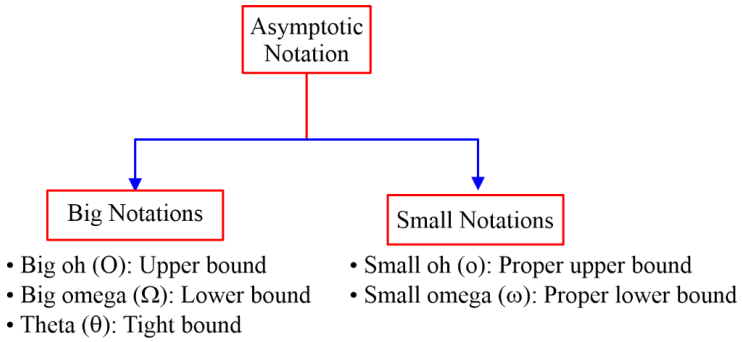
Average case can be calculated form best case to worst case.

1.6 Asymptotic Notations

Suppose, T(n) be a function of time for any algorithm.



1.7 Types of Asymptotic Notations



1.7.1 Big O – Notation

Two Functions $f(n), g(n)$

$f(n) = O(g(n))$

When the growth of $g(n)$ is same or higher than $f(n)$ like $a \le b$

Example:

$f(n) = 3n + 10, g(n) = n^2 + 2n + 5$

$f(n) = O(g(n))$

1.7.2 Ω - Notation

$f(n) = \Omega(g(n))$

$\therefore f(n) \ge C \cdot g(n) \qquad (a \ge b)$

Example: $3^n = \Omega(2^n)$

1.7.3 θ - Notation

If $f(n) \le g(n)$

And

$f(n) \ge g(n)$

$f(n) = g(n)$

$\therefore \quad f(n) = \theta(g(n))$

Example:

$f(n) = 2n^2, g(n) = n+10$

$f(n) > g(n)$ here

so, $f(n) = \Omega(g(n))$ or $g(n) = O(f(n))$

1.7.4. Properties with respect to asymptotic notations

	Reflexive	Symmetric	Transitive	Transpose symmetric
Big oh (O)	✓	×	✓	✓
Big omega (Ω)	✓	×	✓	✓
Theta (θ)	✓	✓	✓	×
Small oh (o)	×	×	✓	✓
Small omega (ω)	×	×	✓	✓

Example 1. Consider the following function

$$f(n) = \sum_{p=1}^{n} p^3 = q$$

Which of the following is/are true for 'q'

(a) $\theta(n^4)$ (b) $\theta(n^5)$ (c) $O(n^5)$ (d) $\Omega(n^3)$

Solution: (a, c, d)

$$f(n) = \sum_{P=1}^{n} P^3$$

$$= 1^3 + 2^3 + 3^3 + 4^3 \ldots\ldots\ldots\ldots + n^3$$

$$= \left(n\left(\frac{n+1}{2}\right)\right)^2$$

$$= O(n^4) \text{ or } \Omega(n^4)$$

$$= \theta(n^4)$$

Example 2. Consider the following functions:

$$f(n) = \sum_{P=1}^{n} P^{1/2} = q$$

Find the value of q in terms of asymptotic notation.

Solution: $f(n) = \sum_{P=1}^{n} P^{1/2}$

$$= 1 + (2)^{1/2} + (3)^{1/2} + \ldots\ldots$$

$$= \frac{2}{3}\left[n^{3/2} - 1\right]$$

$$= \frac{2}{3}n^{3/2} - \frac{2}{3}$$

$$= O(n^{1.5})$$

$$= O(n\sqrt{n})$$

Example 3. Arrange the following functions in increasing order.

$$f_1 = n\log n,\ f_2 = \sqrt{n},\ f_3 = 2^n,\ f_4 = 3^n,\ f_5 = n!,\ f_6 = n^n,\ f_7 = \sqrt{\log n},\ f_8 = 100n\log n$$

$$\rightarrow f_7 < f_2 < fl = f_8 < f_3 < f_4 < f_5 < f_6$$

Example 4. Arrange the following functions in increasing order.

$f_1 = 10,\ f_2 = \sqrt{n},\ f_3 = \log\log n,\ f_4 = (\log n)^2,\ f_5 = n^2$

$f_6 = n\log n,\ f_7 = n!,\ f_8 = 2^n,\ f_9 = n^n,\ f_{10} = n^2\log n$

$\rightarrow\ f_1 < f_3 < f_4 < f_2 < f_6 < f_5 < f_{10} < f_8 < f_7 < f_9$

Example 5. Arrange the following functions in increasing order.

$f_1 = \log\log n$ $\quad$ $f_9 = n\log\log n$

$f_2 = \log n$ $\quad$ $f_{10} = n^2\log n$

$f_3 = (\log n)^2$ $\quad$ $f_{11} = n^3$

$f_4 = \sqrt{\log n}$ $\quad$ $f_{12} = 2^n$

$f_5 = n^{1/10}$ $\quad$ $f_{13} = e^n$

$f_6 = n$ $\quad$ $f_{14} = n!$

$f_7 = n^2$ $\quad$ $f_{15} = n^n$

$f_8 = n\log n$ $\quad$ $f_{16} = n^{3/2}$

$f_1 < f_4 < f_2 < f_3 < f_5 < f_6 < f_9 < f_8 < f_{16} < f_7 < f_{10} < f_{11} < f_{12} < f_{13} < f_{14} < f_{15}$

$a^{\log_b c} \Leftrightarrow c^{\log_b a}$

$\therefore 2^{\log_2 n} \Leftrightarrow n^{\log_2 2} = n$

Example 6. Arrange the following functions in increasing order.

$f_1 = n!,\ f_2 = n^n$

$f_1 = n \times (n-1)(n-2) \times \ldots \times 3 \times 2 \times 1$

$f_2 = n \times n \times n \times n \times \ldots \times n \times n \times n$

$f_2 > f_1$

$\therefore f_1 = O(f_2)$

$\boxed{2^n < 3^n < 4^n < n! < n^n}$

Question.

Which of following is TRUE?

(1) $2^{\log_2 n} = O(n^2)$ TRUE

(2) $n^2 \cdot 2^{3\log_2 n} = O(n^5)$ TRUE

(3) $2^n = O(2^{2n})$ TRUE

(4) $\log n = O(\log\log n)$ FALSE

(5) $\log\log n = O(n\log n)$ TRUE

Solution:

(1) $2^{\log_2 n} = O(n^2)$

$= n^{\log_2 2}$

$= n$

$= n = O(n^2)$

(2) $n^2 \cdot 2^{3\log_2 n} = O(n^5)$

$= n^2 \cdot n^{3\log_2 2}$

$= n^2 \cdot n^3$

$= n^5$

$= n^5 = O(n^5)$

(3) $2^n = 2^{2n}$

$2^n = 2^n.2^n$

$2^n \le 2^{2n}$

$2^n = O(2^{2n})$ True

(4) $\log n > \log\log n$

$\log n \neq O(\log n)$

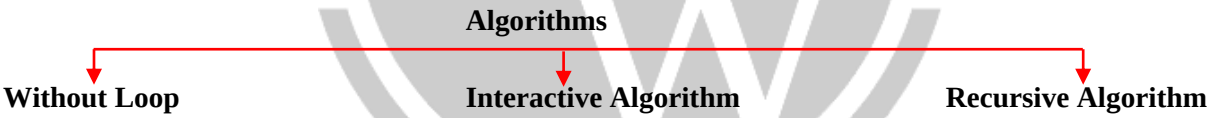
False

(5) $\log\log n \le n\log n$

$\log\log n = O(n\log n)$

True

1.8. Analysis of an Algorithm



1.8.1 Without loop

Example:

```
int fun (in + n)
{
    return n*(n+1)/2;
}
```

Solution.

Here 1 multiply, 1 division, 1 addition

∴ O (1) [no loops, no recursion]

1.8.2. Iterative Algorithm Analysis

Example 1:

```
for (i =1; i ≤ n; i=i*2)
printf("Sushil")
```

Solution.

$i=1, 2, 2^2, 2^3 ..., 2^k$

$\rightarrow$

$2k \leq n$

$k \log 2 \leq \log n$

$k \leq \frac{\log n}{\log 2}$

$\therefore k \leq \log_2 n$

$k = \lfloor \log_2 n \rfloor$

So, this will execute $\lfloor \log_2 n \rfloor + 1$ time and Complexity O ($\log_2 n$)

Example 2:

```
For (i=1; i ≤ n; i=i*3)
printf("Aaveg");
```

Solution.

So, this will execute $\lfloor \log_3 n \rfloor + 1$ time and complexity O ($\log_3 n$)

- $i = 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow ... \rightarrow n$
  $i = n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow ... 1$

Example 3:

```
for (i = 1; i ≤ n; i++)
    {
        for (j=1; j ≤ 10; j++)
            {
                printf("Dhananjay");
            }
        }
```

Solution.

This will execute $10 \cdot n$ times and complexity O(n)

Example 4:

```
for (i = 1; i <= n; i = i*3)
    for (j = 1; j ≤ n; j++)
        printf("Prapti");
```

Solution.

Total $n\left(\lfloor \log_3 n \rfloor + 1\right)$ time execute and Complexity = O ($n \log_3 n$)

1.8.3. Recursive Algorithm Analysis

Example 1:

```
void fun (in + n)        T(n)
{
    if (n > 0)           1 compare; C1 time
    {
        if ("% d", n);   ←  C2 time
        fun (n - 1);     ←  T(n-1)
    }
}
```

Let T(*n*) be the Complexity time taken by algo for n size i/p

Solution.

$T(n) = C_1+C_2+T(n-1)$

$T(n) = T(n-1) + C$ $\quad n > 0$

$T(0) = C$ ← Constant

$T(n) = C;$ $\quad n = 0$

$T(n) = T(n - 1) + C;$ $\quad n > 0$

Example 2:

```
    void fun (in + n)        T (n)
{
    if (n > 0)   ← C1 time
    {
        for (i = 1; i <= n; i + 1)  ← n time
        printf("Hello");
        fun (n - 1);   ← T (n - 1)
    }
}
```

Solution.

$T(n) = C_1 + n - 1 + T(n - 1)$

$= T(n - 1) + n$ $\quad n > 0$

$T(0) = C$ $\quad n = 0$

Example 3: ←

```
void fun (in + n)          T(n)
{
 if (n > 0)   ← C1
   {
    for (i = 1; i < = n; i = i*2)  ← ⌊log2 n⌋
```

```
        printf("Divyajyoti");
    fun (n - 1);  <-------  T (n - 1)
    }
}
```

Solution. $T(n) = T(n-1) + O(\log_2 n); \quad n > 0$

or

$T(n) = T(n-1) + \log_2 n$

$T(0) = C$ } or $n = 0$

$T(0) = O(1)$ }

1.9 Solving Recurrence Relation

1.9.1 Substitution Method

Example: (1)

$$T(n) = T(n-1) + C$$
$$T(1) = C$$

n size on problem $n-1$ size x_1 convert them

$T(n) = T(n-1) + C$

$\downarrow$

$[T(n-2) + C] + C$

$T(n) = T(n-2) + 2C$

$\downarrow$

$= T(n-3) + 3C$

$T(n) = T(n-k) + kC$

$\therefore n - k = 1$

$T(n) = T(1) + (n-1)C$

$= C + (n-1)C$

$T(n) = O(n)$

Example (2)

$$T(n) = T(n-1) + C \cdot n$$
$$T(1) = C$$

Solution.

$\therefore T(n) = T(n-1) + C \cdot n$

$= [T(n-2) + C \cdot (n-1)] + C \cdot n$

$= [T(n-3) + C \cdot (n-2)] + C(n-1) + C \cdot n$

$= T(n-3) + (n-2) \cdot C + (n-1) \cdot C + n \cdot C$

$= T(n-k) + C(n-k+1) + C(n-k+2) + \ldots + C(n-k+k)$

$\therefore n - k = 1$

$T(n) = T(1) + T(2) + C(3) + C(4) + \ldots + C(n-1) + C(n)$

$= C + C(2) + (3)C + 4(C) + \ldots + (n-1)C + (n) \cdot C$

$= C[1 + 2 + 3 + \ldots + n]$

$= C \cdot n \frac{(n+1)}{2}$

$= O(n^2)$

Example (3)

$T(n) = T(n/2) + C$

$T(1) = 1$

Solution.

$T(n) = T(n/2) + C$

$= [T(n/2^2) + C] + C$

$= T(n/4) + 2C$

$= T(n/2^3) + 3C$

$T(n) = T(n/2^k) + kC$

$= (n/2^k) = 2$

$T(n) = T(2) + (\log_2 n - 1) C$

$= 1 + (\log_2 n - 1) C$

$= O(\log n)$

Example (4)

$T(1) = 1$

$T(n) = 2T(n/2) + C$

Solution. $T(n) = 2\left[2T\left(\frac{n}{2^2}\right) + C\right] + C$

$= 2^2 T\left(\frac{n}{2^2}\right) + 2^2 C + C$

$= 2^2\left[2T\left(\frac{n}{2^3}\right) + C\right] + 2C + C$

$= 2^3 T\left(\frac{n}{2^3}\right) + 2^2 C + 2C + C$

$= 2^k T\left(\frac{n}{2^k}\right) + 2^{k-1} C + 2^{k-2} C + \ldots + 2^1 \cdot C + C$

$\frac{n}{2^k} = 1 \quad \therefore n = 2^k$

$\rightarrow T(n) = nT(1) + 2^{k-1} \cdot C + 2^{k-2} \cdot C + \ldots + 2C + C$

$= 2^k + 2^{k-1} \cdot C + 2^{k-2} + \ldots + 2C + C$

$= 2^k + C(2^{k-1} + 2^{k-2} + \ldots + 2^1 + 2^0)$

$= 2^k + C \frac{(2^k - 1)}{2-1}$

$= 2^k + C(2^k - 1)$

$= 2^k + 2^k \cdot C - C$

$= n \cdot C$

$= O(n)$

1.9.2 Master's Method

$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k (\log n)^p)$

$a \geq 1, b > 1, k \geq 0$, p = real number

If $a > b^k$ or $\log_b a > k$

$T(n) = \Theta\left(n^{\log_b a}\right)$

Question 1. $\mathbf{T(n) = 2T\left(\frac{n}{2}\right) + (n)^0 \log n}$

Solution. $a = 2, b = 2, k = 0$

$a > b^k; 2 > 2^0; 2 > 1$

$T(n) = \Theta(n)$

Question 2. $\mathbf{T(n) = 2T\left(\frac{n}{2}\right) + n}$

Solution. $a = 2, b = 2, k = 1, p = 0$

$T(n) = \Theta(n \cdot \log n)$

Question 3. $\mathbf{T(n) = 2T\left(\frac{n}{2}\right) + n \log n}$

Solution. $a = 2, b = 2, k = 1, p = 1$

$\therefore T(n) = \Theta(n (\log n)^2)$

(b) If $p < 0$ then $T(n)$

$T(n) = O(n^k)$

Question 4. $T(n) = T\left(\frac{n}{2}\right) + C$

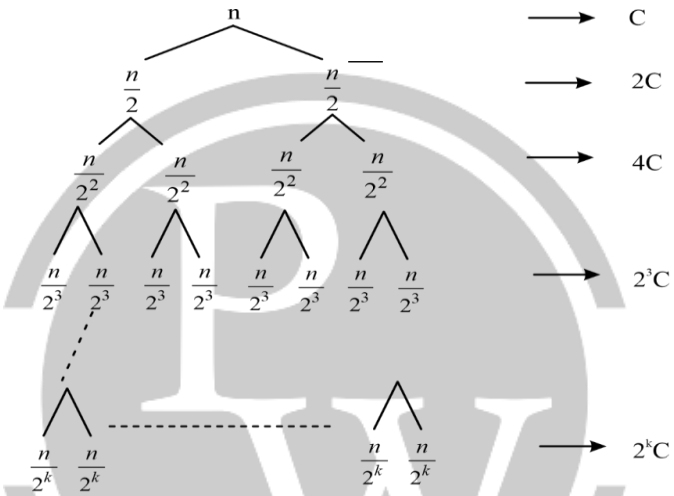
Solution.

$T(n) = \Theta(n^2 \log n)$

1.9.3. Recursive Tree

(1) $T(n) = 2T\left(\frac{n}{2}\right) + C$

$T(1) = C$



$\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$

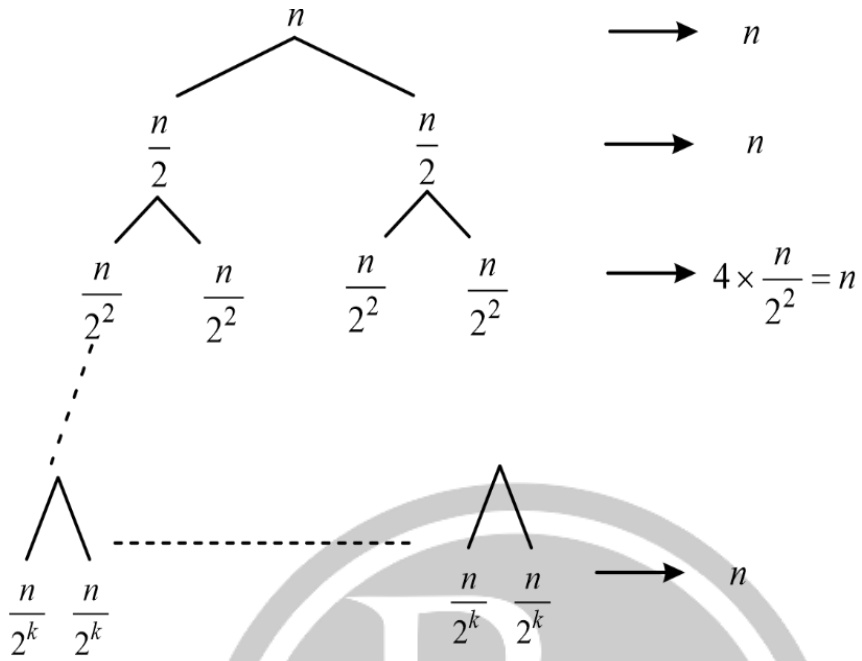
Total Work done $= C + 2C + 2^2C + 2^3C + \ldots + 2^kC$

$= C(1 + 2 + 2^2 + \ldots + 2^k)$

$= c\left(\frac{2^{k+1} - 1}{2 - 1}\right)$

$= C(2^{k+1} - 1$
$= C(2 2^k - 1)$
$= C(2n - 1)$
$= O(n)$

(2) $T(n)=2T\left(\frac{n}{2}\right)+n$



$\frac{n}{2^k}-1; n-2^k; k-\log_2 n$

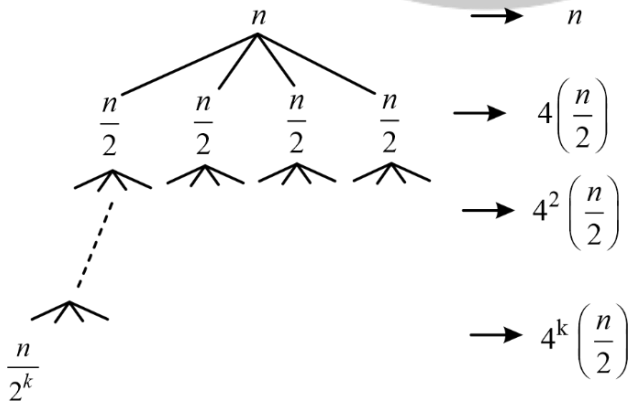
$\therefore n + n + n + \ldots + n$

$= \text{k } n$

$= \text{n } n\log_2 n$

$= O(n\log_2 n)$

(3) $T(n)=4T\left(\frac{n}{2}\right)+n$



$n = 2^k$, $k = \log_2 n$

$= n+4\left(\frac{n}{2}\right)+4^2\left(\frac{n}{2}\right)+\ldots+4^k\left(\frac{n}{2}\right)$

$$= n\left[1+2+2^2+2^3+\ldots+2^k\right]$$

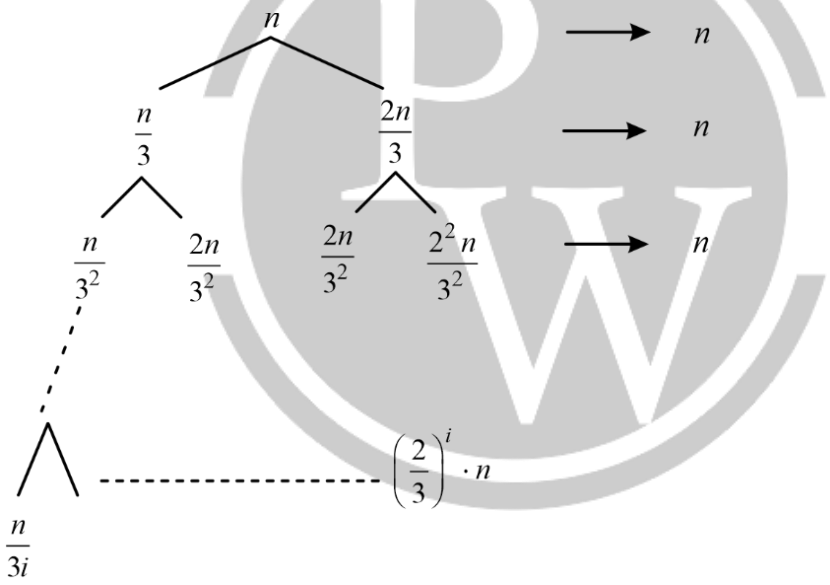
$$= n\left(\frac{2^{k+1}-1}{2-1}\right)$$

$$= n\left(2\cdot 2^{k-1}\right)$$

$$= n(2n)-1$$

$$= O\left(n^2\right)$$

(4) $T(n)=T\left(\frac{n}{3}\right)+T\left(\frac{2n}{3}\right)+n$ T(1) = 1



$\frac{n}{3i}-1;\ n-3i;\ i-\log_3 n$ [3]

$= n + n + \ldots + \log_3 n$ T (n)

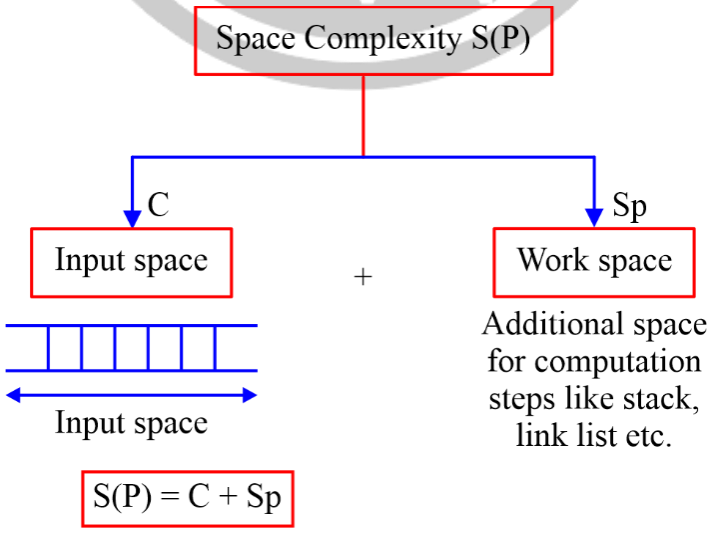
$= (n + n + n + \ldots + \log_3 n) \geq n + \ldots + \log_3 n$

$\Omega(n \log_{3/2} n)$

1.10 Recurrence Relations and their Time Complexity

$T(n) = C; n = 2$ $T(n) = 2\,T(\sqrt{n}) + C; n > 2$	O (logn)
$T(n) = C; n = 2$ $T(n) = T(n-1) + C; n > 2$	$O(n)$
$T(n) = C; n = 1$ $T(n) = T(n-1) + n + C; n > 2$	$O(n^2)$
$T(n) = C; n = 1$ $T(n) = 2T(n-1) + C; n > 1$	$O(2^n)$
$T(n) = C; n = 1$ $T(n) = 2T\left(\frac{n}{2}\right) + C; n > 1$	$\theta(n)$
$T(n) = C; n = 1$ $T(n) = 2T\left(\frac{n}{2}\right) + n; n > 1$	$\theta(n\log n)$
$T(n) = C; n = 1$ $T(n) = T\left(\frac{n}{2}\right) + C; n > 1$	$\theta(\log n)$
$T(n) = 1; n = 2$ $T(n) = T\left(\sqrt{n}\right) + C; n > 2$	$\theta(\log\log n)$

1.11 Space Complexities



```
Int n, A[n];
Algorithm Rsum(A, n)
{
  if (n = 1) return (A(1));
  else;
  return (A[n] + RSum(A, (n–1));
 }
```

- **Time Complexity** = O(n)
- **Space Complexity**
- We need stack space
- Stack is used to store activation records of function calls
- Size of activation records is trivial
- Stack size that we need = O(n)
- Space complexity = O(n)

```
Algorithm A(n)
{
   if (n = 1) return;
   else;
 {
   A(n/2);
  }
}
```

Recurrence relation

$$T(n) = C;\ n = 1$$

$$T(n) = T\left(\frac{n}{2}\right) + C;\ n > 1$$

Time Complexity = O(log n)

Space Complexity

- Space complexity will depend on number of activation record pushed into the stack

Suppose, n = 16

A (1)
A (2)
A (4)
A (8)
A (16)

For $n = 2^K$ we are pushing the 'K' activation record

∴ **Space Complexity**

$$n = 2^K$$
$$\log n = K\log_2 2$$
$$K = \log_2 n$$
$$\boxed{\text{Space Complexity} = O(\log n)}$$

Example 3

```
Algorithm A(n)
{
  if (n = 2) return;
  else;
  return (A √n );
}
```

Solution:

$$T(n) = 1;\ n = 2$$
$$T(n) = T\left(\sqrt{n}\right) + C;\ n > 2$$

Time Complexity = O (loglogn)

Space Complexity

Suppose n = 16

A(1)
A(2)
A(4)
A(16)

∴ For $2^{n/2^k}$ manner we are pushing in stack

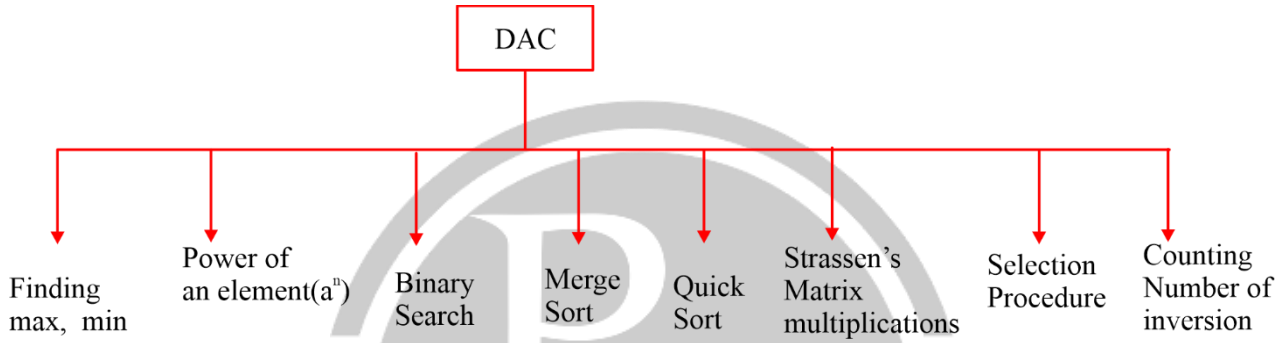
$$2^{n/2^k} \geq 2$$
$$\frac{n}{2^k}\log_2 2 \geq \log_2 2$$
$$n \geq 2^K$$
$$K \leq \log_2 n$$

$$\boxed{\text{Space complexity} = O\left(\log_2 n\right)}$$

❑❑❑

2 DIVIDE AND CONQUER

2.1 DAC Application



2.2 Finding Maximum Minimum element

Recurrence Relation:

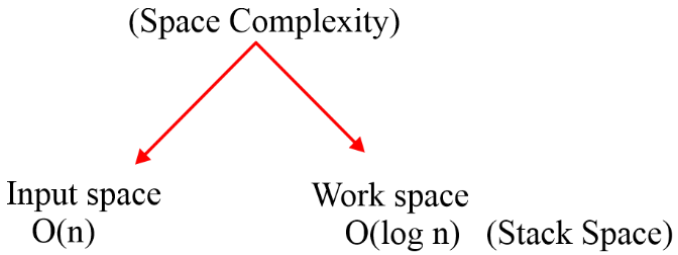
$$T(n)=\begin{cases}1 \text{ if } n=1 \text{ or } n=2\\ 2T\left(\frac{n}{2}\right)+1;\ n>2\end{cases}$$

Time Complexity:

$$T(n)=O(n)$$

- Time complexity is same for every case (Best case/Worst case).

Space Complexity:



$$\text{Space Complexity} = O(n) + O(\log n)$$
$$= O(n)$$

Number of comparisons to find maximum / minimum element on an given array of n elements:

$$\text{Comparison} = \frac{3n}{2} - 2$$

2.3 Power of an Element

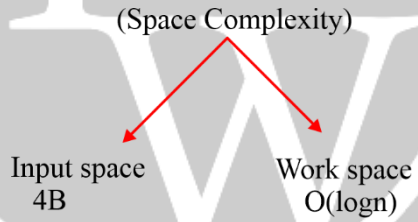
Recurrence relation:

$$T(n) = \begin{cases} 1 \text{ if } n = 1 \\ T\left(\frac{n}{2}\right) + 1;\ n > 1 \end{cases}$$

Time Complexity:

$$T(n) = T\left(\frac{n}{2}\right) + C$$
$$T(n) = O(\log n)$$

Space Complexity:



Space Complexity = 4B + O(logn)

= O(logn)

Number of multiplications to find a^n

Multiplication = O(logn)

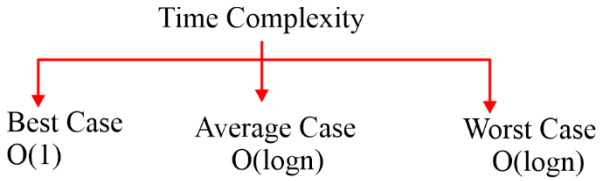
2.4 Binary Search

Given a sorted array and an element x, need to return the index of element x if it is present then 1, otherwise – 1.

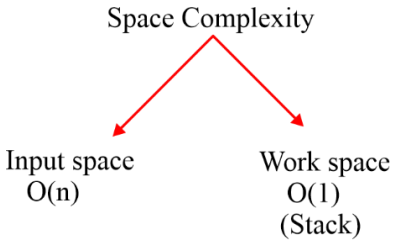
Recurrence relation:

$$T(n) = \begin{cases} 1; & n = 1 \\ T\left(\frac{n}{2}\right) + C;\ n > 1 \end{cases}$$

Time Complexity:



Space Complexity:



Space complexity = O(n) + O(1)

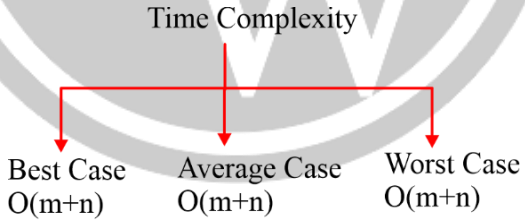
= O (n)

2.5 Merge Algorithm

- Merging two sorted sub arrays of input size m,n.
- Number of comparisons to merge two sorted sub arrays of size m,n.

Comparisons = m + n – 1 (worst case)

Number of moves =m + n (Outplace Algorithm)



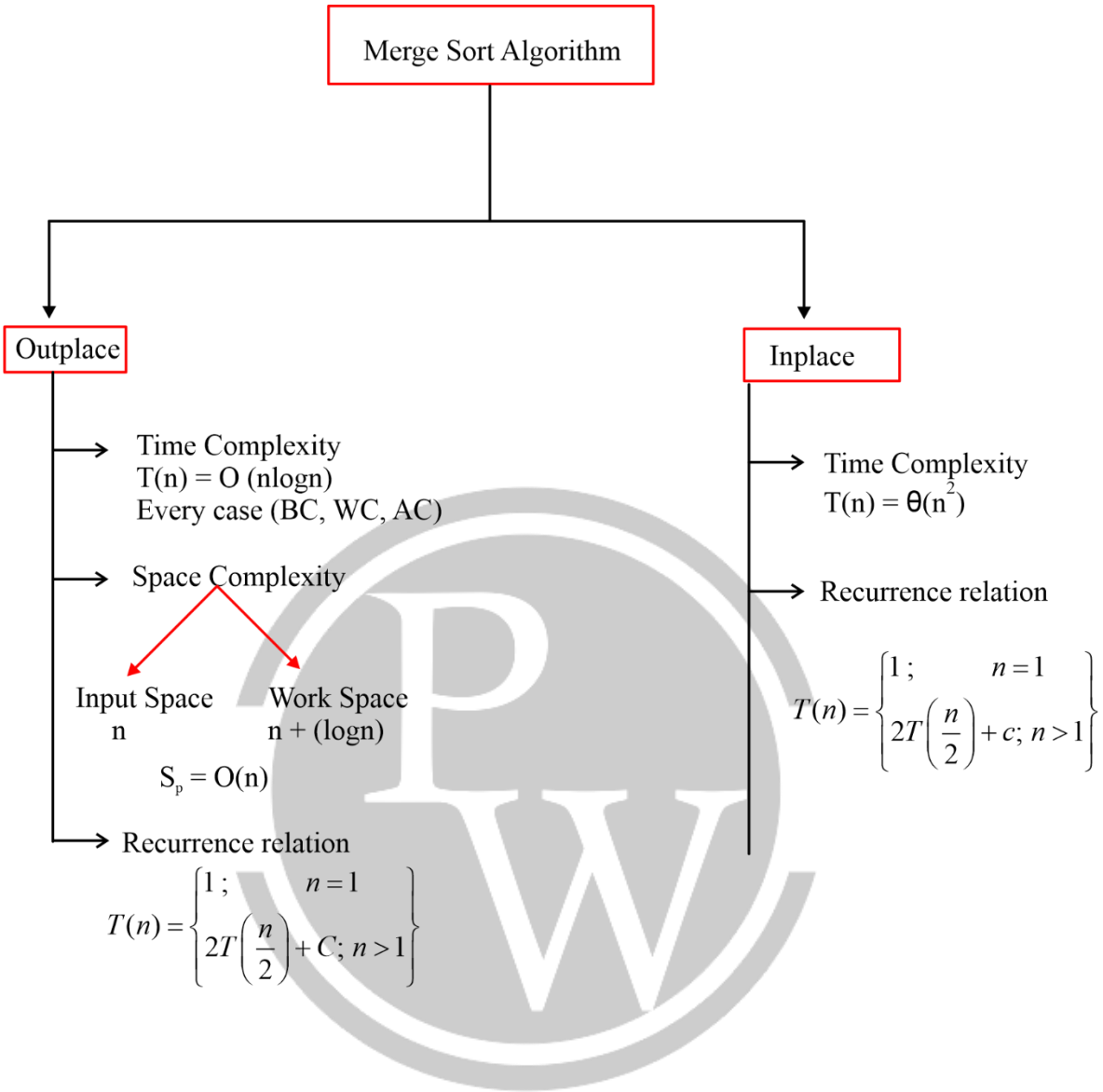
Number of comparisons in best case of merging two sorted subarrays of size m, n.

comparisons = min (m, n)

Moves = m + n (Always)

Note:
Best Case comes in comparisons no effect on moves.

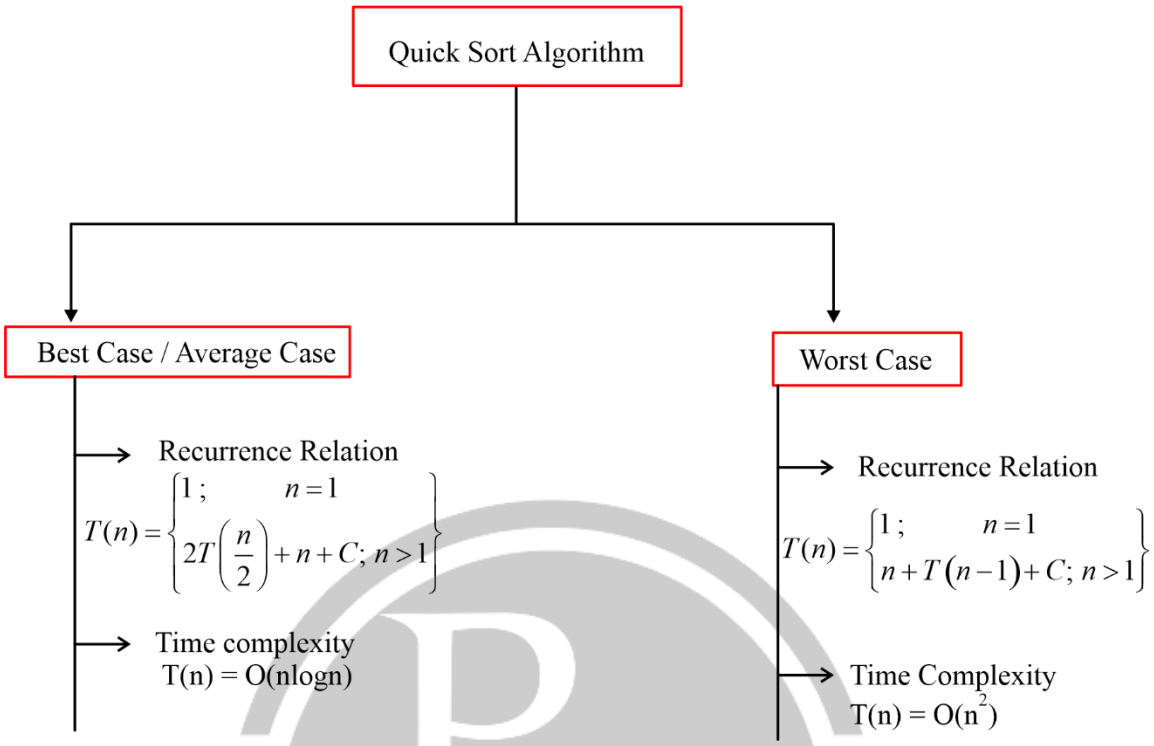
2.5.1 Merge Sort Algorithm:



Note:

- In GATE exam if merge sort given then always consider outplace.
- If array size very large, merge sort preferable.
- If array size very small, then prefer insertion sort.
- Merge sort is stable sorting technique.

2.6 Quick Sort Algorithm



Example 1: In Quick for sorting n elements, the $\left(\frac{n}{16}\right)^{th}$ smallest element is selected as pivot. what is the worst-case time Complexity?

Solution.

$$T(n) = T\left(\frac{n}{16}\right) + T\left(\frac{15n}{16}\right) + O(n)$$

= (solve by recursive tree method)

Example 2: The median of *n* elements can be found in O (n) time then, what is the time complexity of quick sort algo in which median selected as pivot?

Solution.

$T(n) = O(n) + C + O(n) + T(n/2) + T(n/2)$

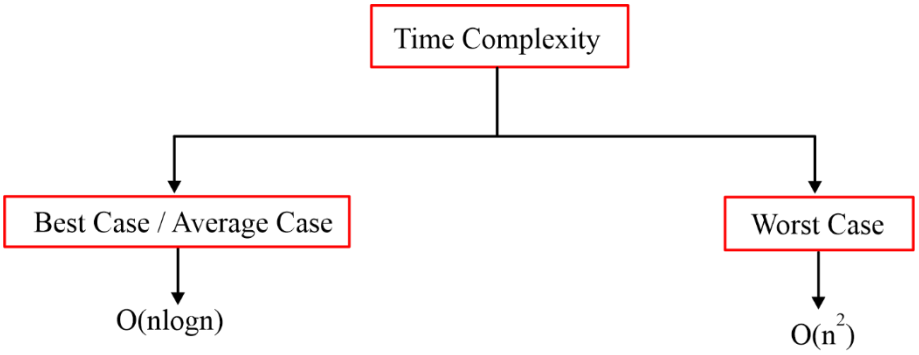
Find median — swap median with last — Partition algo

$= 2T(n/2) + C \cdot n$

$= O(n \log n)$

2.6.1 Randomized Quick Sort

- In Randomized quick short algorithm selection of pivot element can be taken randomly.



2.7 Counting Number of Inversion

- Counting number of inversion on given an array of an element.

  Time complexity T(n) = O(nlogn)

2.8 Selection Procedure

Find K^{th} smallest on given an array of an element and integer K.

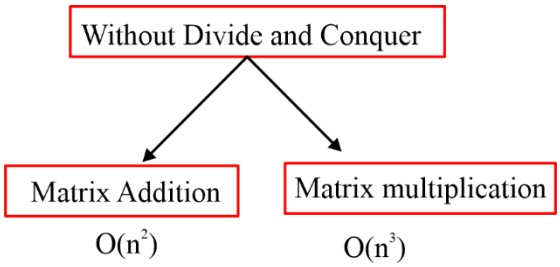
Time Complexity:

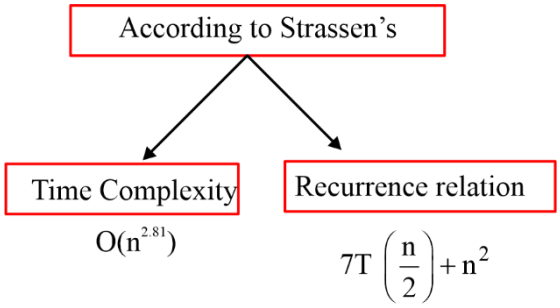
$T(n) = O(n^2)$

Space complexity:

Space Complexity = O(n)

2.9 Strassen's matrix Multiplication





2.10 Comparison Based Sorting Algorithms

Sorting Algorithm	Basic logic of sorting Algo	BC	AC	WC	Stable sorting	Inplace sorting
Quick sort	Choose pivot element place in correct position	$\theta(n\log n)$	$\theta(n\log n)$	$\theta(n^2)$	No	Yes
Merge sort	Divide to equal parts recursively sort each sub part & marge them	$\theta(n\log n)$	$\theta(n\log n) = n \log n$	$\theta(n\log n) = n \log n$	Yes	No
Heap sort	Build heap(max) delete max place	$\theta(n\log n)$	$\theta(n\log n)$	$\theta(n\log n)$	No	Yes
Bubble sort	Compare exchange	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	Yes	Yes
Selection sort	Find position of min element from [1 to n]	$\theta(n^2)$	$\theta(n^2)$	$\theta(n^2)$	No	Yes
Insertion sort	Insert a [i + 1] into correct position	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	Yes	Yes

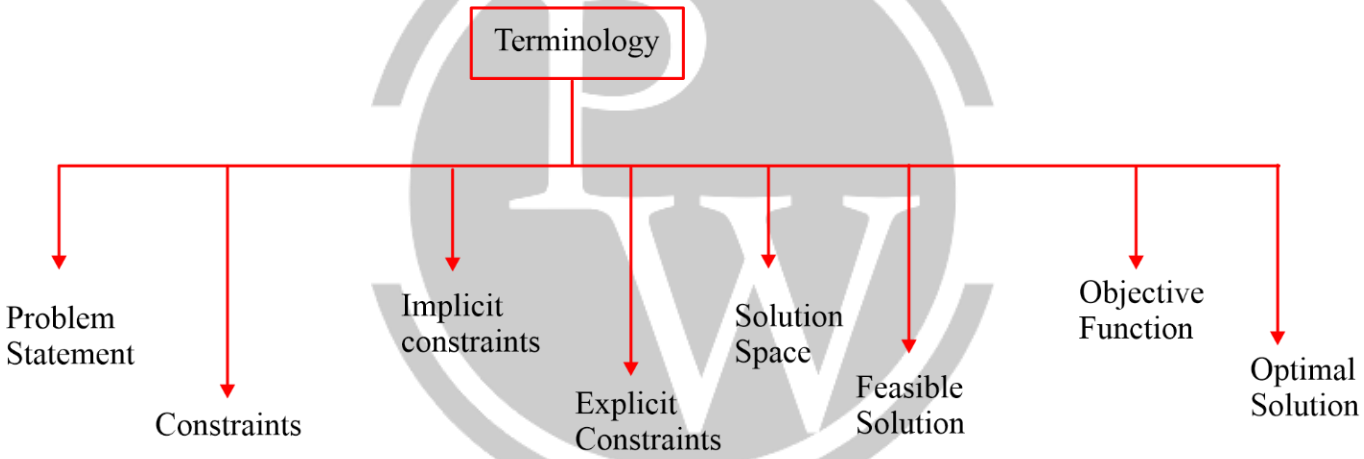
❑❑❑

3 GREEDY TECHNIQUE

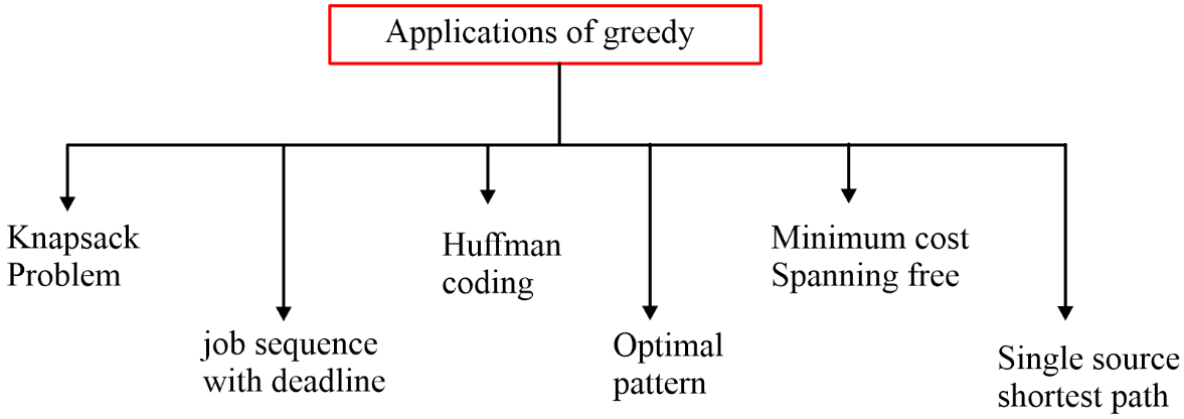
3.1 Greedy Technique

- Greedy method is an algorithm design strategy used for solving problems where solution are seen as result of making a sequence of decisions.
- A problem may contain more than one solution.

3.2 Terminology



3.3 Applications of greedy

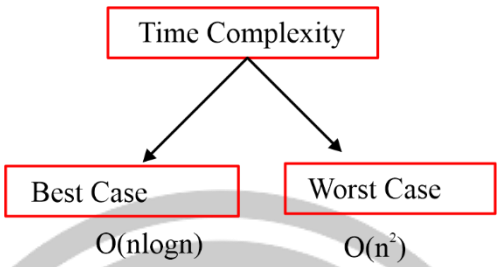


3.4 Knapsack Problem

Time complexity T(n) = O(nlogn)

3.5 Job Sequence with Deadline

- Single CPU only.
- Arrival time of each job is same.
- No pre-emption.



3.6 Optimal Merge Pattern

- This is a problem related to merging of files. Given a set of n-files in sorted order. It is required to merge them into a single sorted file with 2-way merging.
- This problem is like merging process in merge sort. In merge sort we were interested in number of comparisons but in optimal merge pattern we are interested in record movement (i.e moving a record from one file to another file).
    - If file F1 has 'n' records and file 'F2 ' has 'm' records then number of record movement will be 'm+n'. 1 2

  The problem of optimal merge pattern involves merging of n-files (n≥2).
- At any point choose two records with least weight merge them and put them in list and continue it until all records are merged.
- Time complexity T(n) = O(nlogn)
- Space complexity = O(n)

3.7 Huffman Coding

- Huffman coding is essentially a non-uniform encoding with convention that the character with higher frequency (probability) of occurrence will be enclosed with less number of bits.
- It comes under data compression technique.
- Time complexity T(n) = O(nlogn)

3.8 Minimum Cost Spanning Tree

3.8.1 Graph

Graph (V., E)

Set of vertices set of edges

- Let G(V, E) be a simple graph then

$$\text{Maximum edges} = \frac{V(V-1)}{2}$$

$$E \leq \frac{V(V-1)}{2}$$

$E \leq C.V^2$ C is constant

Note:

$E = O(V^2)$

$\text{Log } E = O(\log V)$

3.8.2 Graph Representation

Graph Representation

Adjacency matrix Adjacency list

- For more edges (Dense Graph) Adj. matrix is better (density more).
- For less edge (sparse graph) Adj list is better.

	Matrix	List
(1) Finding degree of vertex⇒ Time Complexity	O(V) Every Case	O(1) Best Case O(V1) Worst Case
(2)Finding total edges ⇒ Time Complexity	$O(V^2)$ Every Case	O(V+2E) Worst Case O(V) Best Case
(3) Finding 2-vertices adjacent (or)not ⇒ Time Complexity	O(1)	O(V-1) Worst Case O(1) Best Case
(4) G(V,E) ⇒ space	$O(V^2)$ Every Case	O(V+E) Every Case

3.8.3 What is Spanning Tree

A subgraph T(V, E') of G(V, E) where E' is the subset of ($E' \subseteq E$) is a spanning tree iff 'T' is a tree.
A sub graph G('V, E') of G(V, E) is said to be spanning tree.
(1) T' should contain all vertices of G
(2) T' should contain (V-1) edged where V is number of vertices without cycle.

(3) T' should connected.

3.8.4 Minimum Cost Spanning Tree

Minimum cost spanning tree is the one in which cost of the spanning tree formed should be minimum.

3.8.5 Prims Algorithm

- Select Any vertex

  Time complexity = V + VlogV + 2E + ElogV

  = O(E + V)logV

Using Sorted Array & Adjacency List

V + 2E + E × V = O(EV)

Using Sorted Array & Adjacency List

V × O(1) + V2 + E × V= O(EV)

3.8.6 Kruskal algorithm

- Take first minimum edge

  Time complexity = E log E + (V + E)

  = O(E log E) = O(ElogV)

  If edges are already sorted

  TC = O(E + V)

3.9 Single Source Shortest Path

3.9.1 Dijkstra Algorithm

- Using min heap & adjacency list = O(E + V)logV
- Using adjacency Matrix & Min heap = $O(V^2 ElogV)$
- Using adjacency list & Unsorted Array = $O(V^2)$
- Using adjacency list & Sorted Doubly Linked List = O(EV)

3.9.2 Bellman-Ford

- Time Complexity = O(EV)
- If negative edge weight cycle then for some vertices Incorrect answer.

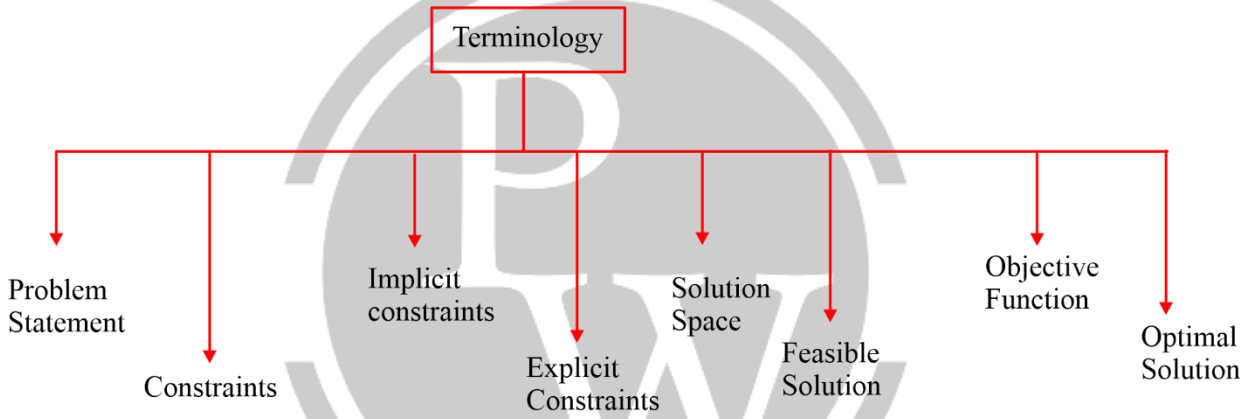
❑❑❑

4 DYNAMIC PROGRAMMING

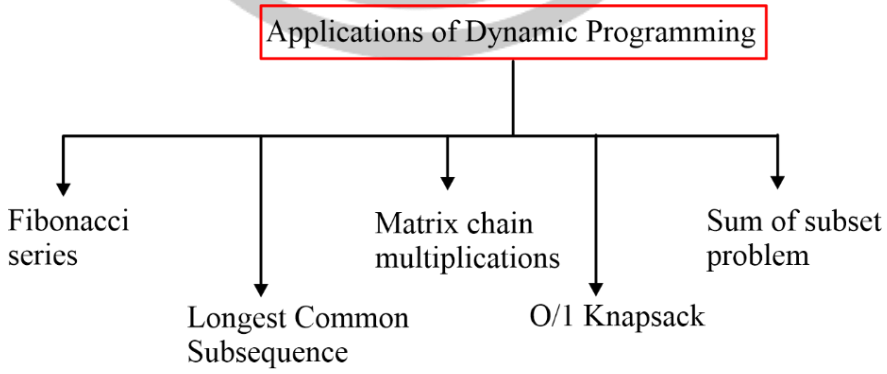
4.1 Dynamic Programming

In dynamic programming for optimal solution always computes distinct function calls.

4.2 Terminology



4.3 Application of Dynamic Programming

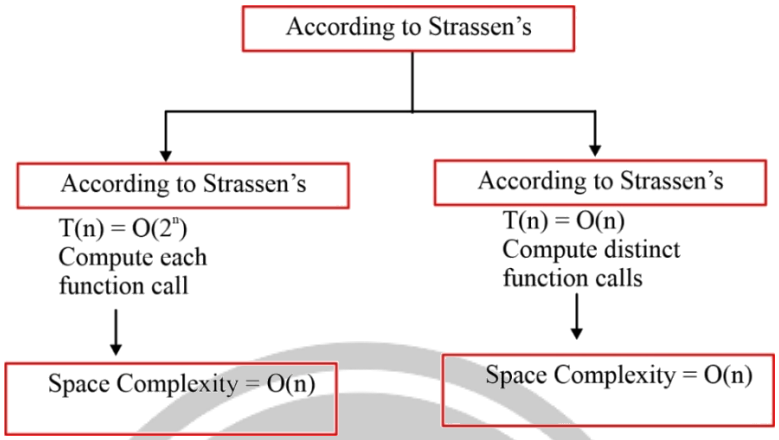


4.4 Fibonacci Series

- Time complexity T(n) = O(nlogn)
- Computes distinct function calls.

4.5 Job Sequence with Deadline

- Single CPU only
- Arrival time of each job is same
- No pre-emption



4.6 Longest Common Sub sequence (LCS)

- For common subsequence always consider two strings:
- P = <ABCDB> – Q = <BDCABA>
- Common subsequences for both 'p' are
- S = <A>
- S = <AB>
- S = <CAB>
- S = <BDAB>
- A common subsequence of longest length is known as longest common subsequence.
- For above problem longest common subsequence will be of length u.

4.6.1 Applications of LCS

1. Genomics
2. Software engineering applications
3. Plagiarism
4. Data gathering system of search engines

4.6.2 Algorithm for LCS

```
LCS (p,q)
{
1.  For   i ←0 to n −1
          L[i −1]=0
2.  For   j ←0 to m −1
          L[−1, j]=0
```

3. *For* $i \leftarrow 0$ *to* $n-1$

   For $j \leftarrow 0$ *to* $m-1$

   If $(p[i]=q[j])$ *then*

   $L[i, j]=1+L(i-1, j-1]$;

   else

   $L[i, j]=max\{L[i, j-1], L[i-1, j]\}$

   }

- Time complexity of step 1 = O(n)
- Time complexity of step 2 = O(m)
- Time complexity of step 3 = O(mn)
- Total Time complexity = O(n) + O(m) + O(mn)
  = O(mn)
- Space complexity = O[(M+1).(n+1)]
  = O(mn)

4.7 Matrix Chain Multiplications

Two matrices 'A' and 'B' are compatible if and only number of column of first matrix must be equal to number of rows of second matrix.

4.7.1 Brute force method

Number of parenthesizing for a given chain is given by Catalan number: $\left[\frac{1}{n+1}\ {}^{2n}C_n\right]$

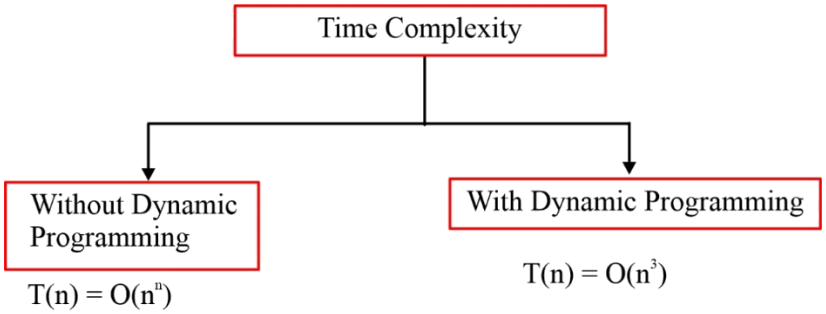
Time complexity = $O(n^n)$

Space complexity = O(n)

4.7.2 Algorithm For Matrix Chain Multiplication

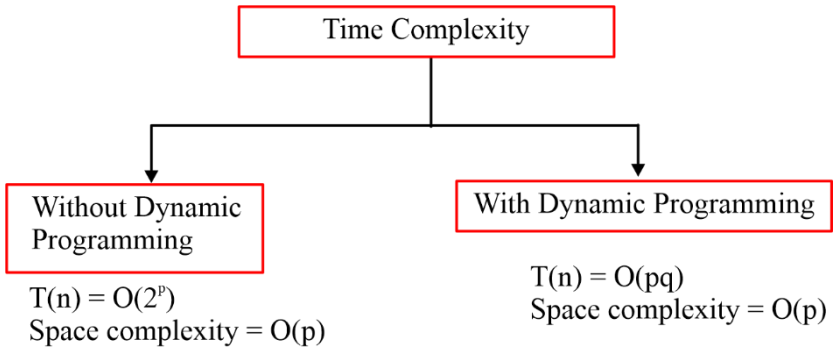
The time complexity of multiply the given chain of n matrices <A_1, A2 A3…. An> using dynamic programming (district function call) is $O(n^3)$

Space complexity = $O(n^2)$



4.8 O/1 Knapsack Problem

The maximum profit can be achieved by O/1 knapsack problem where capacity of problem is 'p' and number of objects are 'q'.



4.9 Sum of Subset Problem

- Given n-elements and an integer 'm', it is required to determine whether there exists a subset of given n elements, whose sum equal M.
- This is a decision problem (True/False).

4.9.1 Algorithm for Sum of Subset Problem

```
SoS(n, M, A)
// A [1 . . . n] is an array of elements
    {
1.  for i = 0 to n
        for j = 0 to M
        if (i > =0 and j = 0)
        SoS [i, j] = T
    else
        if (i = 0 and j > 0)
        SoS [i, j] = F;
    else
        if (A[i] > j)
        SoS [i, j] = SoS [i – 1, j]
    else
        SoS [i, j] = SoS [i – 1, j] or
        SoS [i – 1, j – A[i]]
    }
```

4.9.2 Time Complexity of SoS

Two for loops are there thus repeating for (n * m) times. Thus, time complexity = O(n * m)

Time complexity of SoS becomes exponential if $M = 2^n$

$$T.C = O(n \times 2^n)$$

❐❐❐

GATE 2027 CS — Chapter 5: DSA-IV: Graphs — BFS, DFS, MST & Shortest Paths

Syllabus §4(graphs) + §5(graph traversals, MST, shortest paths). 6–8 marks most years: BFS/DFS orders, Dijkstra traces, MST facts.

One habit carries this entire chapter: **keep a visited set and a distance label on every node**. Every GATE graph question is a 3–6 node toy graph; simulate the algorithm with a pen and the marks are yours.

5.1 Vocabulary & counting — the marks that take 20 seconds

■ THE BOOKISH CORE (what the textbook says)

- Simple undirected graph: max edges **n(n–1)/2** (complete graph K_n). Directed: max **n(n–1)**.
- **Handshaking lemma:** Σ degrees = 2E (undirected); Σ in-degree = Σ out-degree = E (directed). Odd-degree vertices are always EVEN in number.
- **Tree** = connected + acyclic ⇔ n vertices and n–1 edges. A forest with c components has n–c edges.
- **Euler circuit** (uses every edge once, returns): connected + every vertex even degree. **Hamiltonian cycle** (visits every vertex once): no simple criterion — NP-complete — but in K_n there are **(n–1)!/2** distinct ones.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **From IIT Madras's own GATE 2019 paper:** "Let G be an undirected complete graph on n vertices, n > 2. The number of different Hamiltonian cycles in G is…" → **(n–1)!/2**. Fix vertex 1, permute the other n–1 around the circle (n–1)! arrangements, divide by 2 because each cycle reads the same forwards and backwards. Madras loves exact counting — expect another one in 2027.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Counting "paths between two vertices" by multiplying degrees — valid only in specific tree/complete-graph patterns; in general it's a DFS counting problem. GATE's attractive wrong answers come from degree multiplication.

5.2 Two ways to store a graph

Representation	Space	Edge lookup	Best for	GATE usage
Adjacency matrix	O(V²)	O(1)	Dense graphs (E ≈ V²), Floyd-Warshall	"Is (u,v) an edge?" instant
Adjacency list	O(V + E)	O(degree)	Sparse graphs (E ≈ V), BFS/DFS/Dijkstra	The default GATE structure

Rule of thumb: GATE's toy graphs are small enough to store either way; the question is what the ALGORITHM needs — iterating over all neighbours (list wins) or testing arbitrary pairs (matrix wins).

5.3 BFS — the ink drop (level = distance)

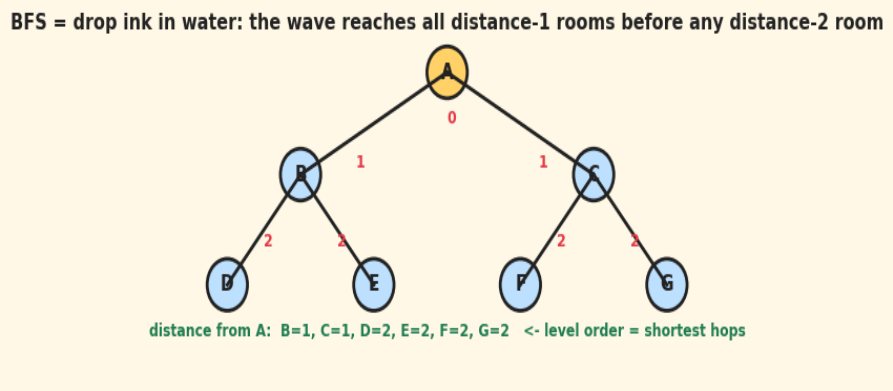


Fig 5.1 — BFS from A: the wave visits every distance-1 vertex before any distance-2 vertex.

■ **THE BOOKISH CORE (what the textbook says)**

- BFS uses a **QUEUE**: dequeue u, enqueue unvisited neighbours, mark visited. O(V + E).
- **On unweighted graphs, BFS order IS shortest-path distance.** That is the single most tested fact about BFS.
- BFS tree = the union of edges by which vertices were first discovered. Different neighbour orderings → different BFS trees, but the same distances.
- BFS 2-coloring tests bipartiteness: if a vertex ever meets an already-colored neighbour of its own color → odd cycle → NOT bipartite.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- GATE pattern: "The BFS traversal of this graph starting at S, visiting neighbours in alphabetical order, is…" → simulate: queue contents step by step. 2 marks for 90 seconds of queue bookkeeping.
- "Which vertices are at distance exactly 2 from S?" → run BFS levels; count them. GATE asks this nearly every cycle.

```
BFS(S):
dist[S]=0; queue={S}
while queue not empty:
u = dequeue()
for each neighbour v of u (in given order!):
if v unvisited: dist[v]=dist[u]+1; enqueue(v)
```

5.4 DFS — the maze explorer

Real life: DFS is exploring a cave with a ball of string. You unroll the string going deeper and deeper down one tunnel; only when a tunnel dead-ends do you rewind to the last junction and try the next opening. The string is the **recursion stack**.

■ **THE BOOKISH CORE (what the textbook says)**

- DFS uses a **STACK** (explicit, or the call stack via recursion). O(V + E). Goes DEEP before going WIDE — the opposite of BFS.
- **Edge classification (directed):** tree edges (discovery), back edges (to an ancestor IN PROGRESS — gray), forward edges (to a finished descendant), cross edges (between finished subtrees).
- **Cycle detection:** a directed graph has a cycle ⇔ DFS finds a BACK EDGE. In undirected graphs: a cycle ⇔ a non-tree edge to an already-visited vertex that is not your immediate parent.
- **Topological sort:** order vertices by DECREASING finish time in DFS (or: insert each vertex at the FRONT of a list when it finishes). Only works on DAGs — a back edge = cycle = no topo order.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

- GATE pattern: "The number of connected components in this graph" → one DFS per unvisited vertex; count launches. O(V+E).
- "Which of the following is a valid topological order of this DAG?" → check every edge goes left-to-right in your candidate. The all-edges-forward test is 10 seconds per option.

```
DFS(u): // recursion = the ball of string
mark u visited; time++
for each neighbour v of u:
if v unvisited: parent[v]=u; DFS(v) // tree edge
else if v in progress: BACK EDGE -> cycle! // gray vertex
finish[u] = time++; // topo order = decreasing finish
```

5.5 Minimum Spanning Tree — cable the villages for minimum money

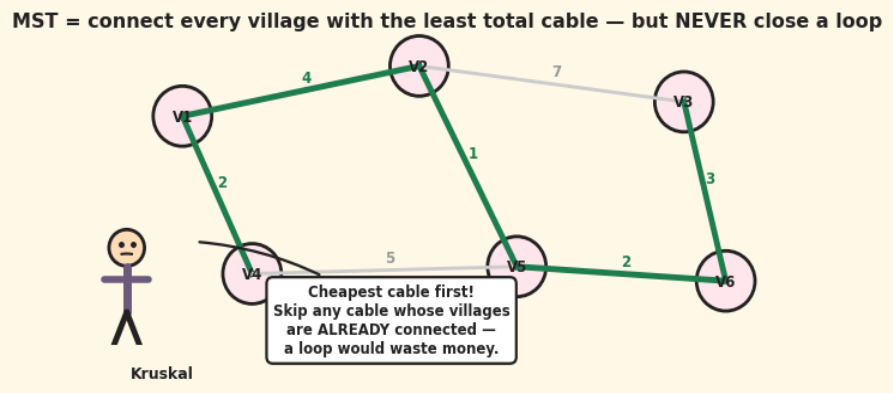


Fig 5.2 — MST keeps only cheapest connecting cables; any cable forming a loop is refused.

■ THE BOOKISH CORE (what the textbook says)

- **Cut property (the whole religion):** for any cut of the graph, the minimum-weight edge crossing the cut belongs to SOME MST. All MST algorithms are this one idea with different bookkeeping.
- **Kruskal:** sort edges by weight, add edge if its endpoints are in DIFFERENT components (union-find). O(E log E). Refuses cycle-closing edges — Fig 5.2.
- **Prim:** grow one tree from a start vertex, always add the cheapest edge leaving the tree (priority queue). O(E log V).
- **Distinct weights → unique MST.** Equal weights → possibly many; every MST still has the same total weight.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE pattern: "Which edge must be present in EVERY MST?" → an edge that is the unique minimum across SOME cut (or the unique max on no cycle — equivalently, its endpoints' only cheap connection). "May be in some MST but not all?" → tie situations.
- "The weight of the MST of this graph is…" → run Kruskal by hand on a 6–8 edge graph; add accepted weights. 2 marks, pure bookkeeping.

```
Kruskal on the Fig 5.2 weights (2,1,2,4,5,3,7):
accept V1-V4(2)? YES accept V2-V5(1)? YES accept V5-V6(2)? YES
accept V3-V6(3)? YES -> 4 edges connect 5 villages: DONE
skip V1-V2(4), V4-V5(5), V2-V3(7) <- they would close loops
MST weight = 2+1+2+3 = 8
```

5.6 Shortest paths — three algorithms, three moods

Algorithm	Edge weights	Idea	Time	When GATE uses it
BFS	All 1 (unweighted)	Levels = distance	O(V+E)	Min hops / edges count
Dijkstra	Non-negative	Greedy: settle nearest unsettled, relax its edges	O(E log V)	Trace distance table on a small graph
Bellman-Ford	Any (incl. negative)	Relax ALL edges V−1 times	O(VE)	Negative weights / detect negative cycle
DAG relax	Any (incl. negative), acyclic	Relax edges in topological order	O(V+E)	One-pass shortest path on a DAG
Floyd-Warshall	Any, no negative cycle	dp: via-k paths, k=1..V	$O(V^3)$	All-pairs, tiny graphs, transitive closure

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **The single most repeated GATE graph task: trace Dijkstra.** Worked example (graph: A–B 4, A–C 1, C–B 2, C–D 5, B–D 1):

```
step settle A B C D (from A)
0 A 0 4 1 inf
1 C 0 3! 1 6 relax C->B: 1+2=3 < 4
2 B 0 3 1 4! relax B->D: 3+1=4 < 6
```

```
3 D FINAL: B=3, C=1, D=4
```

One settled vertex per row, each row relaxes only that vertex's outgoing edges. GATE gives 4 options of final distance arrays — the row-by-row table takes 60 seconds and cannot be wrong. Practice it until it's muscle memory.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- **Dijkstra with negative edges can produce WRONG answers** — after a vertex is settled it never reopens. GATE asks "does Dijkstra work here?" as a 1-mark trap.
- Bellman-Ford: V–1 rounds guarantee correctness IF no negative cycle. After round V–1, run one more pass — any edge still relaxing proves a negative cycle. GATE asks both facts.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- ■ **Probable GATE 2027 (IIT-Madras style):** a counting twist — "The number of spanning trees of a complete graph on n labelled vertices is n^(n–2)" (Cayley). For n = 5 → 125. Madras papers (2011, 2019) repeatedly paired one algorithmic trace with one clean counting fact; Cayley's formula is exactly that species.

Rapid-fire quiz — 10 questions, then check answers

```
1. Edges in K_8? Directed simple graph max on 8 vertices?
2. Graph with Σ degrees = 30 → edges?
3. BFS on unweighted graph gives what?
4. DFS finds which edge type ⇒ cycle (directed graph)?
5. Valid topological order test?
6. Distinct edge weights → how many MSTs?
7. Kruskal's underlying data structure?
8. Dijkstra with a negative edge — safe?
9. Bellman-Ford: rounds? negative-cycle test?
10. Spanning trees of labelled K_5 (Cayley)?
```

■ ANSWERS (cover this box, attempt first!)

- **1.** K_8: 28. Directed: 56.
- **2.** E = 15. (Number of odd-degree vertices is always even.)
- **3.** Shortest-path distance (in hops) from the source to every vertex.
- **4.** A back edge to an in-progress (gray) vertex.
- **5.** Every edge goes forward in the order (tail before head).
- **6.** Exactly one. All MSTs (if many) share the same total weight.
- **7.** Union-Find (disjoint set) for cycle tests.
- **8.** No. Settled vertices never reopen → wrong distances possible.
- **9.** V–1 relaxation rounds; a relaxing edge in round V proves a negative cycle.
- **10.** 5^(5–2) = 125.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Max edges	Undirected n(n–1)/2, directed n(n–1). Σdeg = 2E; odd-degree vertices even in count.
BFS	Queue. Levels = shortest hops on unweighted graphs. 2-color → bipartite test.
DFS	Stack/recursion. Back edge = cycle. Topo sort = decreasing finish time.
MST	Cut property rules all. Kruskal = sort + union-find. Prim = grow cheapest boundary edge.
MST uniqueness	Distinct weights → unique MST. Every MST: same total weight.
Dijkstra	Non-negative only. Settle nearest, relax neighbours. Trace = one row per settled vertex.
Bellman-Ford	Relax all edges V–1 times. Handles negatives. Extra relaxing pass = negative cycle.
DAG shortest path	Relax edges in topo order — one pass, works with negatives.
Floyd-Warshall	$O(V^3)$, all pairs, dp via-k. Also transitive closure (OR instead of min+).

Topic	The one truth GATE keeps asking
Euler vs Hamilton	Euler: all even degree. Hamilton in K_n: (n–1)!/2 — GATE 2019 (IIT-M).

Say "continue" for Chapter 6: Algorithms — sorting, hashing, master theorem, and greedy (Huffman, activity selection, optimal merge).

GATE 2027 CS — Chapter 6: Algorithms: Complexity, Sorting, Hashing & Greedy

Syllabus §5 minus graphs (Ch. 5): asymptotics, the sorting table GATE expects you to recite, hashing probe counts, master theorem, and the greedy gallery.

This chapter is pure marks-per-hour: almost everything here is a table, a formula, or a 60-second simulation. Total ≈ 6–8 marks a year, and the questions recycle the same half-dozen facts.

6.1 Asymptotics — the language of "how slow?"

■ THE BOOKISH CORE (what the textbook says)

- **O** = upper bound (worst case), Ω = lower bound, Θ = both. GATE mostly wants O and Θ.
- Ladder to memorise: **1 < log n < √n < n < n log n < n² < n³ < 2■ < n!**. Anything below 2■ is "poly-time"; 2■ and n! explode.
- Rules: drop constants (3n = Θ(n)); keep the dominant term; log bases are irrelevant (log■n = Θ(log■■n)).
- log grows slower than ANY positive power of n: log n = O(n^0.0001). GATE ranking questions live on this fact.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- GATE pattern: "Arrange in increasing order of growth: n^(1/2), 2^n, n log n, log n" → log n, √n, n log n, 2^n. 1 mark, 10 seconds.

6.2 Master theorem — the divide-&-conquer calculator

■ THE BOOKISH CORE (what the textbook says)

- For T(n) = aT(n/b) + f(n), compare f(n) with **n^(log_b a)**:
- **Case 1:** f(n) grows polynomially slower → T(n) = Θ(n^(log_b a)) [e.g., T(n)=2T(n/2)+n is the BOUNDARY case below]
- **Case 2:** f(n) = Θ(n^(log_b a)·log^k n) → T(n) = Θ(n^(log_b a)·log^(k+1) n) [merge sort: k=0 → Θ(n log n)]
- **Case 3:** f(n) grows polynomially faster AND af(n/b) ≤ cf(n) (regularity) → T(n) = Θ(f(n)) [T(n)=2T(n/2)+n² → Θ(n²)]

```
T(n) = 2T(n/2) + n -> n^log2(2)=n, f=n case 2 -> Θ(n log n) [merge sort]
T(n) = 4T(n/2) + n -> n^2 > n case 1 -> Θ(n²) [naive karatsuba]
T(n) = T(n/2) + 1 -> log_b a = 0, f = 1 case 2 -> Θ(log n) [binary search]
T(n) = 3T(n/2) + n -> n^(lg 3) ≈ n^1.58 case 1 -> Θ(n^1.58) [karatsuba multiply]
```

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- T(n) = 2T(n/2) + n log n: f is not polynomially bigger — regularity case 3 FAILS here; the real answer is Θ(n log² n). GATE loves planting this exact recurrence in options.

6.3 The sorting table — recite it in your sleep

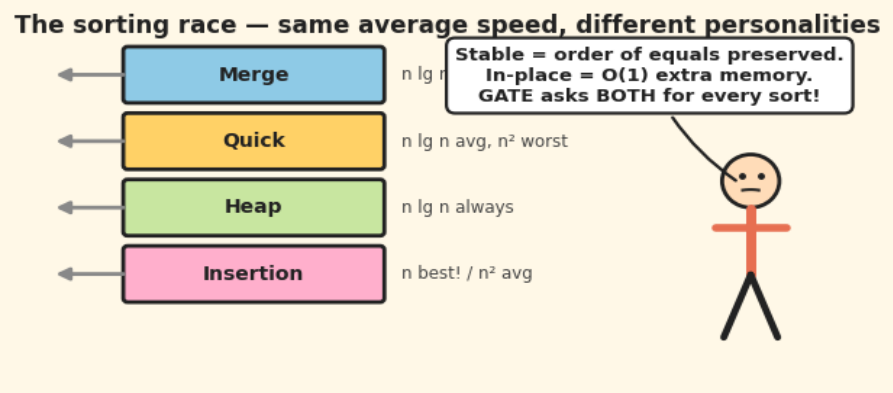


Fig 6.1 — Same average class, different stability, memory, and worst-case stories.

Sort	Best	Average	Worst	Extra space	Stable?	One-line identity
Insertion	O(n)	O(n²)	O(n²)	O(1)	YES	Adaptive: nearly-sorted ≈ linear; great for tiny arrays

Sort	Best	Average	Worst	Extra space	Stable?	One-line identity
Selection	O(n²)	O(n²)	O(n²)	O(1)	NO	Always n–1 swaps — minimal writes
Bubble	O(n)	O(n²)	O(n²)	O(1)	YES	With swap-flag: adaptive; else a turtle
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	YES	The guarantee; loves linked lists
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	NO	Fastest in practice; worst on sorted+pivot=end
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	NO	Guaranteed, in-place, not stable
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)	YES	Small integer range k
Radix	O(nk)	O(nk)	O(nk)	O(n+k)	YES	Stable subroutine per digit

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• The three facts GATE recycles most: (1) **Any comparison sort needs Ω(n log n)** comparisons — that's why merge/heap/quick can't beat it asymptotically. (2) **Selection sort: exactly n–1 swaps.** (3) **Quicksort worst case O(n²)** happens on already-sorted input with end pivot.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **Counting inversions with merge sort — a GATE classic pattern:** modify merge: when you take the RIGHT element before remaining left run, every remaining left element forms an inversion with it. Runs in Θ(n log n) vs Θ(n²) brute force. Expect it as a 2-mark "what does this modified merge return?"

6.4 Hashing — O(1) with a good excuse

Real life: a hash table is a coat check with 100 hooks. The attendant uses a formula on your ticket number to pick a hook. Two coats landing on the same hook = a **collision**. Two philosophies for fixing it: hang chains off each hook (**chaining**) or walk to the next empty hook (**open addressing**).

■ **THE BOOKISH CORE (what the textbook says)**

• **Load factor α = n/m** (items ÷ slots). Everything in hashing is a function of α — GATE asks these formulas with α given numerically.

• **Chaining** (expected, simple uniform hashing): successful search ≈ **1 + α/2**, unsuccessful ≈ **1 + α**. Never "full" — chains just grow.

• **Open addressing** (linear probing): probe sequences: successful ≈ **½(1 + 1/(1–α))**, unsuccessful ≈ **½(1 + 1/(1–α)²)**. Table must stay α < 1; as α → 1, probes explode — e.g., α = 0.9 → unsuccessful ≈ 50 probes!

• **Primary clustering** = linear probing's disease: consecutive occupied slots merge into mega-blocks. **Quadratic probing** and **double hashing** (step size = second hash h■(k), e.g., h■(k) = 1 + (k mod (m–1))) fix it.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• Numeric pattern: "chaining, m = 10, n = 5 (α = 0.5): expected probes for a successful search?" → 1 + 0.25 = 1.25. "Linear probing, α = 0.5, unsuccessful?" → ½(1 + 1/0.25) = 2.5. GATE swaps the two formulas as bait — know which is which.

• Deletion in open addressing: you can't just blank the slot (it breaks probe chains) — use **tombstones** (lazy deletion). GATE asks this as a 1-mark conceptual.

```
double hashing probe sequence: h(k, i) = (h1(k) + i * h2(k)) mod m
e.g., m=11, h1(k)=k mod 11, h2(k)=1+(k mod 10)
k=27: try (5 + i*8) mod 11 -> 5, 2, 10, 7, ... (never clusters linearly)
```

6.5 Greedy algorithms — take the best bite now

■ THE BOOKISH CORE (what the textbook says)

- Greedy works when the problem has the **greedy-choice property** (a locally optimal pick is globally optimal) + **optimal substructure**. GATE rarely asks for proofs — it asks you to RUN the greedy on small data.
- **Activity selection:** sort by FINISH time, keep an activity if it starts after the last kept one's finish. Maximises the count of non-overlapping activities. Sort key is the whole trick — sorting by start time is wrong.
- **Fractional knapsack:** take items by value/WEIGHT ratio descending; fill the bag, top up with a fraction of the next. O(n log n). (0/1 knapsack needs DP — greedy fails there; GATE asks exactly this contrast.)
- **Job sequencing with deadlines:** sort jobs by profit desc; place each job in the latest free slot ≤ its deadline (DSU or slot array).
- **Optimal merge pattern:** repeatedly merge the TWO SMALLEST files (min-heap). Same skeleton as Huffman.

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **Huffman coding — asked in GATE repeatedly (2004, 2007, 2014…):** given frequencies, repeatedly merge the two least frequent nodes; the weighted path length = total merge cost.

```
frequencies: a:45, b:13, c:12, d:16, e:9, f:5 (per 100 chars)
merge f5+e9=14; merge c12+b13=25; merge 14+d16=30;
merge 25+30=55; merge 45+55=100.
cost = 14+25+30+55+100 = 224 bits for 100 chars (vs 300 fixed-length)
prefix-free: no code is a prefix of another -> uniquely decodable.
```

Notice: merge cost = sum of internal node weights = total bits. One heap, five merges, one formula. GATE's Huffman questions are this exact computation with shuffled numbers.

■■ TRAP ALERT (where rank-hopefuls bleed marks)

- Huffman is optimal AMONG PREFIX codes — if the question says "variable-length codes" without the prefix constraint, the premise changes; GATE words this carefully.
- Greedy on 0/1 knapsack can be arbitrarily bad — a heavy high-ratio item blocks two cheap perfect items. If GATE shows a fractional-style greedy failing, the correct alternative is DP (Chapter 7).

■ WHAT GATE ASKS (patterns from 2007–2026 papers)

- **■ Probable GATE 2027 (IIT-Madras style):** a "compute the quantity" hybrid — e.g., "Using double hashing with h1 and h2 as given, list the first 3 probe positions for keys 38, 91, 50" plus one comparison-ranking MCQ. Madras papers favour exactly this: mechanical computation + one conceptual ranking.

Rapid-fire quiz — 10 questions, then check answers

```
1. Rank: log n, 2^n, n log n, n^(1/2)?
2. T(n) = 2T(n/2) + n?
3. T(n) = 4T(n/2) + n?
4. Quicksort worst case input + pivot choice?
5. Which sorts are stable? Which in-place?
6. Lower bound for comparison sorting?
7. Chaining, α = 2: expected successful probes?
8. Linear probing, α = 0.5: expected unsuccessful probes?
9. Activity selection sort key?
10. Huffman cost for a:5,b:9,c:12,d:13,e:16,f:45?
```

■ **ANSWERS (cover this box, attempt first!)**

- **1.** log n < √n < n log n < 2^n.
- **2.** Θ(n log n) — merge sort (master theorem case 2).
- **3.** Θ(n²) — case 1 (n² dominates n).
- **4.** Sorted/reverse-sorted with last (or first) element pivot → Θ(n²).
- **5.** Stable: insertion, bubble, merge, counting, radix. In-place: insertion, selection, bubble, quick (O(log n) stack), heap.
- **6.** Ω(n log n) comparisons.
- **7.** 1 + α/2 = 2.
- **8.** ½(1 + 1/(1−α)²) = 2.5.
- **9.** Earliest FINISH time — not start time, not duration.
- **10.** Merge pairs: 14,25,30,55,100 → total 224 bits (mirror of the example).

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
Growth ladder	1 < log n < √n < n < n log n < n² < n³ < 2■ < n!.
Master thm.	Compare f with n^(log_b a): slower→case1, equal(×log^k)→case2, faster+regular→case3.
Sort facts	Selection: n−1 swaps always. Quick worst Θ(n²) sorted input. Merge stable O(n) space. Heap in-place unstable.
Comparison LB	Any comparison sort: Ω(n log n). Counting/radix escape it — no comparisons at all.
Hash α=n/m	Chaining: 1+α/2 succ, 1+α unsucc. Open addressing needs α<1; probes blow up near full.
Linear probing	½(1+1/(1−α)) succ; ½(1+1/(1−α)²) unsucc. Primary clustering is its disease.
Double hashing	h(k,i) = (h1(k) + i·h2(k)) mod m — h2 ≠ 0, coprime-ish step kills clustering.
Greedy gallery	Activity: by finish. Fractional knapsack: by ratio. Jobs: profit desc + latest slot. Merge files: two smallest.
Huffman	Merge two least frequent, always. Cost = sum of merge weights = total bits. Optimal PREFIX code.
Greedy vs DP	Greedy: locally-best is globally-best (fractional knap). 0/1 knapsack: greedy fails → DP (Ch. 7).

Say "continue" for Chapter 7: Dynamic Programming — GATE's single highest-weight algorithmic topic.

GATE 2027 CS — Chapter 7: Dynamic Programming — recursion with a notebook

Syllabus §5 "dynamic programming". GATE's favourite design technique: 4–6 marks a year, and every question is a table you already know how to fill.

If Chapter 2's recursion was the leap of faith, DP is the leap of faith PLUS a notebook: **never compute the same subproblem twice.** GATE DP questions are glorified table-filling drills. Learn the patterns below and you own this topic.

7.1 The two ingredients — when DP applies

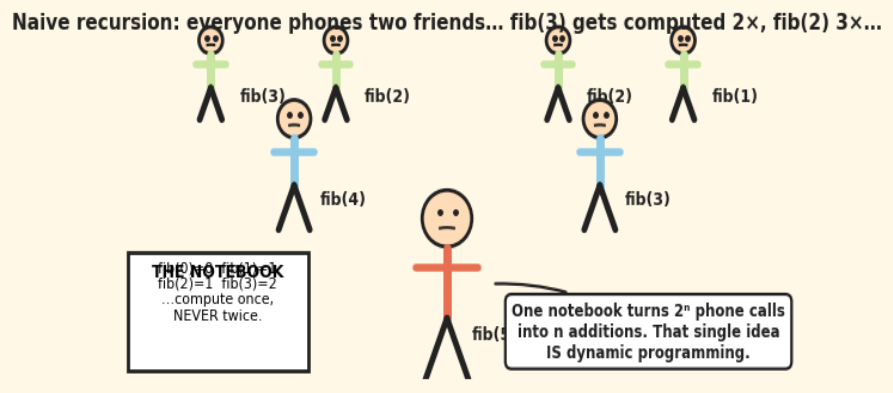


Fig 7.1 — Memoization: compute each subproblem once. That idea alone converts exponential to polynomial.

■ THE BOOKISH CORE (what the textbook says)

- **Overlapping subproblems:** the recursion tree recomputes the same states (fib(3) computed twice, thrice…). If subproblems are all DISTINCT (like plain merge sort), DP buys nothing — divide & conquer already wins.
- **Optimal substructure:** the optimal answer is built from optimal answers of subproblems (shortest path: prefix of a shortest path is shortest). No such structure → DP/greedy both fail (e.g., longest SIMPLE path).
- **Memoization** (top-down: recursion + lookup table) vs **tabulation** (bottom-up: fill table in order). Same complexity; GATE doesn't care which you picture.
- Spotting DP in the exam hall: the words **"number of ways"**, **"minimum/maximum cost"**, **"count the sequences"**, or two strings being aligned/edited. Any of these → think DP table.

7.2 The GATE pattern table — each with its recurrence

Pattern	State / recurrence	Answer	Time
Stairs/ways	dp[i] = dp[i–1] + dp[i–2]	dp[n]	O(n)
Grid paths	dp[i][j] = dp[i–1][j] + dp[i][j–1]	dp[m][n] = C(m+n, n)	O(mn)
0/1 knapsack	dp[i][w] = max(dp[i–1][w], v_i + dp[i–1][w–w_i])	dp[n][W]	O(nW)
Coin change (min)	dp[a] = min(dp[a–c]) + 1 over coins c ≤ a	dp[amount]	O(amount·coins)
Coin change (ways)	dp[a] += dp[a–c] (coins OUTER loop!)	dp[amount]	O(amount·coins)
LCS	match → 1 + dp[i–1][j–1]; else max(dp[i–1][j], dp[i][j–1])	dp[m][n]	O(mn)
Edit distance	match → dp[i–1][j–1]; else 1 + min(del, ins, sub)	dp[m][n]	O(mn)
LIS	dp[i] = max(dp[j]+1) for j smaller with a[j] < a[i]	max dp[i]	O(n²) / O(n log n)
Matrix chain	m[i][j] = min over k: m[i][k] + m[k+1][j] + p_{i–1}·p_k·p_j	m[1][n]	O(n³)

7.3 Worked drills — the exact shapes GATE photocopies

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **0/1 knapsack (W = 5; items (w,v): (2,3), (3,4), (4,5), (5,6)):** best = items 1+2 → weight 5, value **7**. Each table row = "considering first i items"; entry = best value at capacity w. The recurrence's two choices — skip (row above) or take (value + entry at w–w_i) — are the whole table.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **LCS ("ABCBDAB", "BDCABA"):** length **4** (BCBA or BDAB). Diagonal move = match (+1); else max from top or left. Cell (i,j) = LCS of the first i and first j characters.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **Edit distance ("kitten" → "sitting"): 3** = substitute k→s, e→i, insert g. Every mismatch costs 1 more than the best of the three neighbouring cells. This was a genuine GATE question (2008).

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **Matrix chain, dims p = {10, 20, 30, 40} (A1: 10×20, A2: 20×30, A3: 30×40):** (A1A2)A3 = 10·20·30 + 10·30·40 = 6000 + 12000 = **18000**; A1(A2A3) = 20·30·40 + 10·20·40 = 24000 + 8000 = 32000. Optimal = 18000. GATE gives 3–4 matrices and asks the minimum cost or the parenthesization.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **LIS of [10, 9, 2, 5, 3, 7, 101, 18]:** length **4** (2, 3, 7, 18). O(n^2) DP is enough for GATE; the O(n log n) "patience piles + binary search" is the upgrade interviewers love.

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• **Grid paths on a 3×3 board** (corner to corner, only right/down): **C(4,2) = 6**. With one blocked cell: subtract paths through it. GATE asks the blocked-cell variant regularly.

■■ **TRAP ALERT (where rank-hopefuls bleed marks)**

• **Count-ways coin change: the coin loop must be OUTER.** Coins outer, amounts inner → counts combinations (1+2 and 2+1 are the SAME way). Amounts outer → counts permutations. GATE asks this distinction; it changes the answer entirely.

• **0/1 knapsack is O(nW) = pseudo-polynomial** — polynomial in the numeric VALUE of W, exponential in its bit length. GATE asks this exact sentence as a 1-mark conceptual.

7.4 Greedy or DP? — the 10-second decision

Problem	Verdict	Why
Fractional knapsack	Greedy	Ratio sorting provably optimal — best bite now
0/1 knapsack	DP	Greedy can block perfect combos; no greedy-choice property
Activity selection	Greedy	Earliest-finish leaves maximum room — provable
Min coins, arbitrary denominations	DP	Greedy fails for {1,3,4}: amount 6 → greedy 3 coins, optimal 2
Shortest path (no negative cycles)	Greedy (Dijkstra) / DP	Both work; negatives force Bellman-Ford
Longest simple path	Neither	No optimal substructure — NP-hard

■ **WHAT GATE ASKS (patterns from 2007–2026 papers)**

• ■ **Probable GATE 2027 (IIT-Madras style):** a 2-mark table-cell question — "the partially filled LCS table for strings X and Y is given; the value of cell (4,5) is…" or "which parenthesization gives minimum cost for these 4 matrices?" IIT Madras's 2011/2019 CS papers both carried exactly this species of small-table DP. The defence is one: practise filling each table type by hand, once, slowly.

Rapid-fire quiz — 10 questions, then check answers

```
1. The two ingredients for DP?
2. fib(n): naive vs memoized complexity?
```

```
3. Knapsack W=5, items (2,3),(3,4),(4,5),(5,6) → best value?
4. Min coins for amount 6, coins {1,3,4}: greedy vs optimal?
5. Ways to make amount 4 with {1,2,3} (combinations)?
6. Paths on 3×3 grid, right/down only?
7. LCS("ABCBDAB","BDCABA")?
8. Edit distance kitten→sitting?
9. LIS of [10,9,2,5,3,7,101,18]?
10. MCM cost for dims {10,20,30,40}?
```

■ **ANSWERS (cover this box, attempt first!)**

- **1.** Overlapping subproblems + optimal substructure.
- **2.** O(2■) vs O(n) time, O(n) space.
- **3.** 7 (items 1+2).
- **4.** Greedy: 4+1+1 = 3. Optimal: 3+3 = 2. Hence DP, not greedy.
- **5.** 4: 1111, 112, 22, 13.
- **6.** C(4,2) = 6.
- **7.** 4 (BCBA or BDAB).
- **8.** 3 (k→s, e→i, insert g).
- **9.** 4: 2, 3, 7, 18.
- **10.** 18000 via (A1A2)A3.

One-page cheat sheet — stick this on your wall

Topic	The one truth GATE keeps asking
DP trigger words	"Number of ways", "min/max cost", "count sequences", string alignment → DP table.
Notebook rule	Compute each subproblem once. Memo (top-down) or table (bottom-up) — same big-O.
Ways vs permutations	Coins OUTER loop → combinations. Amounts outer → permutations. GATE bait.
Knapsack	dp[i][w] = max(skip, take). O(nW) — pseudo-polynomial, say it proudly in answers.
LCS / edit dist.	Match: diagonal +1 (LCS) / 0 (edit). Else max of top,left (LCS) / 1+min 3 neighbours (edit).
LIS	dp[i] = best earlier smaller + 1. $O(n^2)$; binary-search piles → O(n log n).
MCM	m[i][j] = min over k of m[i][k]+m[k+1][j]+p_{i–1}·p_k·p_j. $O(n^3)$.
Grid paths	dp = top + left. Closed form C(m+n, n); blocked cell → subtract through-cell paths.
Greedy vs DP	Fractional knapsack greedy; 0/1 DP; activity greedy; min-coins DP; longest simple path: nothing.
Exam tactic	GATE DP = table filling. Practise each pattern once by hand → +4 to +6 marks secured.

That completes Syllabus Section 5 (Algorithms). Say "continue" for Batch 3: Engineering Mathematics (Ch 8: Discrete Math logic & proofs → Ch 9: sets/relations/lattices → Ch 10: combinatorics → Ch 11: linear algebra → Ch 12: probability).